Transportation Research Part E 192 (2024) 103752

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

Making better order fulfillment in multi-tote storage and retrieval autonomous mobile robot systems

Zhizhen Qin, Yuexin Kang, Peng Yang *

Division of Logistics and Transportation, Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China
Institution of Data and Information, Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China

ARTICLE INFO

Keywords:
Robotic warehouse system
Multi-tote autonomous mobile robot
Order picking
Joint scheduling

ABSTRACT

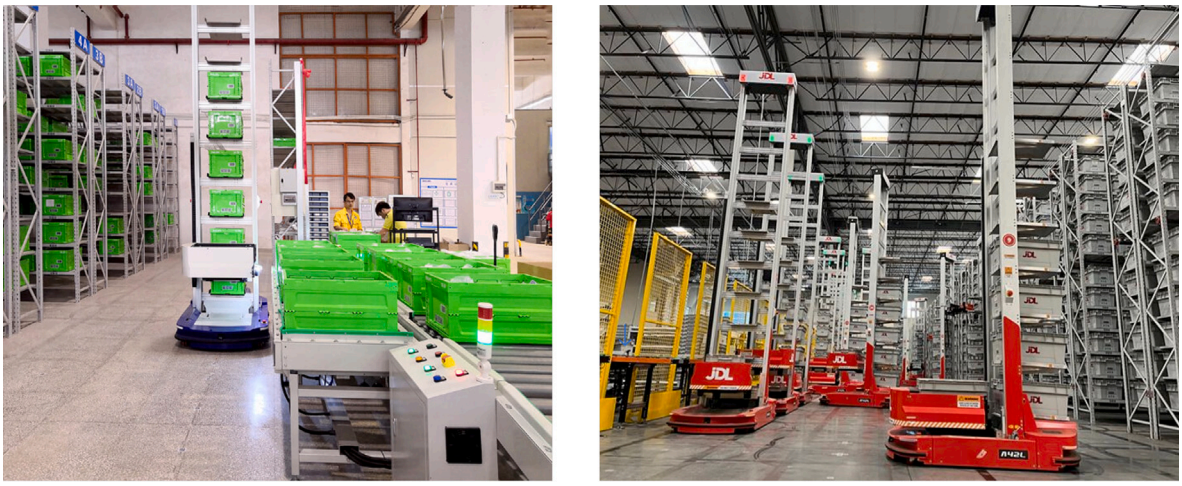
The multi-tote storage and retrieval (MTSR) autonomous mobile robot (AMR) systems are increasingly prominent in e-commerce and third-party logistics. These systems feature robots capable of handling multiple totes per tour. The operational decisions of order fulfillment in MTSR AMR systems include assigning and sequencing orders and totes at various workstations and scheduling robots. The intricate interplay among order, tote, and robot significantly heightens the order fulfillment challenge in MTSR AMR systems. This study proposes a mixed-integer programming model that simultaneously determines the assignment and sequence of orders and totes, and the scheduling of robots in MTSR AMR systems with multiple workstations. We develop an item characteristic-driven adaptive large neighborhood search algorithm tailored to efficiently resolve this multifaceted problem. The numerical experiments demonstrate the effectiveness of the proposed algorithm, which swiftly yields optimal or near-optimal solutions for small-scale instances. For large-scale instances, the algorithm achieves a 50.2% reduction in makespan compared to the scheduling methods currently used in an actual warehouse. Keeping the number of robots fixed and increasing the buffer positions of the robots can lead to a substantial makespan reduction, up to 55.4%. Intriguingly, we find that augmenting the number of workstations does not proportionally decrease the makespan once the capacity of put wall at each workstation surpasses five order boxes. Furthermore, the experiments reveal that the optimal number of orders per wave is around 100, and a wider warehouse layout can reduce the makespan by 26.3% compared to a narrow layout.

1. Introduction

The burgeoning development of e-commerce has led to increased order volumes, necessitating elevated labor levels, continuous operations, and enhanced storage capacity for goods within limited space in the warehouses (Azadeh et al., 2019). In this context, the importance of robotics and automation has been widely recognized. In 2019, Shopify, an online marketplace in Canada, invested $450 million to acquire 6 River Systems, an automation provider, aiming to expand its AI-driven fulfillment network (Protalinski, 2019). Amazon has boosted warehouse productivity and minimized labor requirements by developing proprietary automation solutions via Amazon Robotics (Amazon, 2022). Mckinsey (2021) predicted that these investments are expected to significantly drive industry growth, with the warehouse automation market projected to reach $51 billion by 2030. Coinciding with advancements in electronic information technology and the Internet of Things, diverse automated warehousing systems continue to emerge (Boysen et al., 2023). The shuttle-based storage and retrieval system (SBS/RS) deploys shuttles that move horizontally and lifts that move

* Correspondence to: Shenzhen International Graduate School, Tsinghua University, Shenzhen, Guangdong Province, China.
E-mail addresses: qzz21@mails.tsinghua.edu.cn (Z. Qin), kyx22@mails.tsinghua.edu.cn (Y. Kang), yang.peng@sz.tsinghua.edu.cn (P. Yang).

https://doi.org/10.1016/j.tre.2024.103752
Received 12 March 2024; Received in revised form 6 June 2024; Accepted 27 August 2024
Available online 5 September 2024
1366-5545/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



(a) A household appliance factory (b) A distribution center

Fig. 1. Some warehouse cases using MTSR AMR system (Hairobotics, 2023).

vertically to carry goods (Tappia et al., 2017). The robotic mobile fulfillment system (RMFS) uses automated guided vehicles (AGVs) to transport movable pods to workstations for order picking (Boysen et al., 2017). These systems, in contrast to traditional manual warehouses, where order picking represents over 55% of total costs (Petersen and Schmenner, 1999; De Koster et al., 2007), delegate the labor-intensive tasks such as transporting goods across extensive storage areas to automation equipment, thereby substantially lowering costs and increasing efficiency (Zhuang et al., 2022; Zhen et al., 2023).

Unlike any previous automation system, the innovative multi-tote storage and retrieval (MTSR) autonomous mobile robot (AMR) system, notable for its flexible configuration and clever design, showcases exemplary order fulfillment performance. Comprising multi-tote robots, workstations, and rack shelves (Fig. 1), the system utilizes robots equipped with multiple buffer positions, a gripper, laser navigation, and a camera for precise tote transportation and collision-free movement. Task assignments and routing schedules are wirelessly transmitted from the warehouse management system. Each workstation incorporates a put wall, conveyor, and picker. The put wall stores current order boxes, and the robot delivers totes to the conveyor, where they are unloaded for the picker to complete the order fulfillment process. Aisles allow for the robot's entry and exit for shelf access. Qin et al. (2024) discover that the multi-tote robot can significantly enhance the throughput capacity and reduce throughput time, thereby increasing efficiency and potential revenue. Besides high efficiency, the MTSR AMR system dynamically adjusts the tote load per tour based on incoming orders, enabling quick deployment with minimal investment.

Due to its attractive technical features, the MTSR AMR systems have been widely applied in e-commerce, third-party logistics, and the footwear and apparel manufacturing industries across Europe and China. Robotic warehouse solution providers such as Hairobotics, Geek+, Savoye, Hikrobot, and Quicktron have developed and marketed MTRS AMR systems. Despite its broad adoption, there is a notable gap in the literature regarding optimizing order fulfillment in MTSR AMR systems. The order fulfillment process within a MTSR AMR system is as follows. Firstly, orders arrive and enter into the order pool. Pending orders in the pool are methodically assigned to designated workstations for processing, with fulfillment exclusively taking place at each workstation. Once processed at a workstation, the corresponding order box is placed on the put wall. Given the limited capacity of the put wall, incoming orders are prioritized based on unique item demands, requiring the handling of corresponding totes by robots for fulfillment. Unlike other systems, the MTSR AMR system orchestrates the scheduling of robots to implement a routing scheme, ensuring efficient transport of required multiple totes to the designated workstations during a single tour. Subsequently, the picker selects items from the totes to fill order boxes. Once all items for an order on the put wall are fulfilled, the order box departs the workstation, paving the way for new orders to enter. Note that the completed order boxes can be temporarily placed next to the workstation and then transported to the packaging area in bulk once a certain quantity is reached. This process is iterated until all orders in the order pool are fulfilled. Considering the distinctive features of the MTSR AMR system outlined above, our research focuses on operational optimization in this system, which involves a detailed examination of order and tote assignment, sequencing, and robot scheduling, with a specific emphasis on their joint optimization.

The research questions in this study are:

1. How should orders be assigned to different workstations, and what sequencing strategy should be employed for their entry into these workstations to enhance overall order fulfillment efficiency?
2. What methods can be employed to determine the sequence of tote sets arriving at different workstations, taking into account the order requirements on the put wall? This consideration is essential to guarantee that the transported totes effectively meet the maximum number of order demands.

3. Which scheduling plan is optimal for organizing robots in tote transportation tasks, and how can the routing for each robot tour in retrieving multiple totes be optimized to minimize the makespan for transporting all necessary totes?

Joint order fulfillment optimization in the MTSR AMR system involves three major decisions: order sequencing (including order assignment in multiple workstations), tote sequencing at workstations (including tote assignment in multiple workstations), and robot scheduling (including routing). This challenge, which includes the entire order fulfillment process in the MTSR AMR system, is defined as the multi-workstation order and tote and robot scheduling problem (MOTSRSP).

The main contributions of this study include:

- This paper is among the first to investigate order and tote sequencing, as well as robot scheduling issues within the MTSR AMR system featuring multiple workstations. Compared to the existing studies on the scheduling of RMFS (Zhuang et al., 2022), the complexity of the MOTSRSP in the novel MTSR AMR system is significantly higher, as the robots handle multiple tote transfers simultaneously. We propose a mixed integer programming (MIP) model that amalgamates multiple decisions to formulate the MOTSRSP, focusing on achieving a balanced workload across workstations to minimize the makespan of order fulfillment.
- To address the knotty MOTSRSP, we devise an item characteristic-driven adaptive large neighborhood search (I-ALNS) algorithm. Within this I-ALNS framework, we implement a two-stage heuristic algorithm: the first stage establishes the order and tote sequence by an item similarity and popularity-driven method, while the second stage uses a genetic algorithm to schedule the robots for the efficient handling of these totes. In each iteration, the I-ALNS employs a combination of destroy and repair operations to generate a new order sequence. This process is followed by the repeated application of the two-stage heuristic algorithm to iteratively improve the solution. This I-ALNS is designed for practical implementation in large-scale scenarios, facilitating the rapid identification of high-quality scheduling schemes.
- We conduct a sensitivity analysis varying the number of robots, buffer positions, workstations, the capacity of the put wall, and order characteristics based on large-scale instances. This analysis provides valuable managerial insights regarding the optimal deployment of robots and workstations within the MTSR AMR system.

The structure of this paper is organized as follows: Section 2 provides a review of the literature. Section 3 describes the MOTSRSP through a mixed-integer programming model. Section 4 presents the heuristic algorithms developed for the MOTSRSP. Section 5 conducts numerical experiments to verify the accuracy of the I-ALNS algorithm and presents managerial insights, while Section 6 concludes with discussions on the findings and directions for future research.

2. Literature review

We primarily review the related work on order fulfillment optimization in picker-to-parts warehousing systems and parts-to-picker warehousing systems (including pod-based and tote-based systems). Pod-based parts-to-picker system is exemplified by the RMFS. Tote-based parts-to-picker systems encompass autonomous vehicle storage and retrieval system (AVS/RS), SBS/RS, and AMR-assisted system (Azadeh et al., 2019). We summarize the typical literature in Table 1 to reveal the research gap that we will bridge.

2.1. Picker-to-parts systems

The order fulfillment process in picker-to-parts systems involves order batching and assignment, picker scheduling and picker routing, and so on. On the one hand, some studies concentrate on optimizing specific steps in order fulfillment. Muter and Öncan (2015) address the picker routing in the order batching problem (OBP) and introduce a column generation (CG) algorithm to solve the MIP model accurately. However, the OBP has been proven as an NP-hard problem, which has led to the widespread use of heuristics. Žulj et al. (2018) apply a hybrid metaheuristic to OBP, significantly enhancing the performance of addressing larger-scale instances. On the other hand, a multitude of studies undertake joint optimization of multiple problems in order fulfillment. Aiming to minimize the total tardiness for all orders, Menéndez et al. (2017) address the order batching and sequencing problem (OBSP), employing a metaheuristic based on variable neighborhood search. Meanwhile, Matusiak et al. (2017) also investigate the OBSP using adaptive large neighborhood search (ALNS) algorithm, where they are devoted to minimizing the total order processing time. Furthermore, routing issues are taken into account, and several tailored metaheuristics have been developed to examine the joint challenges of order assignment, batching, and picker routing in a wave-picking warehouse with multiple pickers (Ardjmand et al., 2018). Based on this, Muter and Öncan (2022) explore the integrated order batching, routing, and picker scheduling problem (OBPSP), minimizing total travel time and makespan through the CG algorithm.

2.2. Pod-based parts-to-picker systems

The pod-based parts-to-picker system is known as RMFS (Azadeh et al., 2019). In the context of order fulfillment within the RMFS, numerous studies have adopted the objective of minimizing pod visits. Various studies have explored different facets of warehouse order fulfillment, primarily concentrating on the joint optimization of orders and storage units such as pods, racks, or shelves. These studies commonly utilize a range of heuristic methods to address these challenges efficiently. Boysen et al. (2017) concentrate on the RMFS and make decisions regarding order assignment and sequencing for different picking stations, along with the sequencing of shelves assigned to those stations, enabling the scheduling of over half of the robots compared to the previous

method. Likewise, Xiang et al. (2018) introduce a heuristic to optimize pod and order assignments in the RMFS to minimize pod visits. Further, an MIP model is developed to simultaneously address order sequencing and rack scheduling, and a two-stage heuristic approach is proposed, which indicates that joint optimization yields significantly fewer robot handling tasks compared to the benchmark and separate optimization (Yang et al., 2021). At the same time, Valle and Beasley (2021) extend the problem to multi-workstation scenarios, designing two metaheuristics to address the order assignment, rack assignment, and rack sequencing. Unlike previous studies on joint optimization, a distinguishing feature of Xie et al. (2021)'s work is the ability to split each order among different workstations for fulfillment. This is the first time split orders have been considered in the context of RMFS, potentially revolutionizing the flexibility and efficiency of the picking process. Extending their work, Zhuang et al. (2022) investigate the order assignment and sequencing as well as the rack scheduling involving multiple workstations, considering the workload balancing across workstations and the rack conflicts. Simultaneously, other studies have concentrated on the joint optimization of picking and replenishment decisions in robotic forward-reserve warehouses, and a picking-replenishment synchronization mechanism is proposed, aiming to streamline operations in the robotic forward area (Jiang et al., 2020).

Nevertheless, pod visits serve only as a proxy index to gauge order fulfillment efficiency, while a gap persists between pod visits and the actual order fulfillment performance of RMFS. To bridge this gap, recent studies have gradually shifted their focus toward time-based metrics, such as total order fulfillment time and makespan, as their objective function. Zou et al. (2017) focus on optimizing the assignment of workstations to robots within RMFS used by online retailers and propose a novel assignment rule based on the handling speeds of different workstations, aiming to reduce total retrieval throughput time. Following that, they investigate battery charging and swapping strategies within RMFS (Zou et al., 2018b). By comparing plug-in charging, inductive charging, and battery swapping strategies, they find inductive charging to perform best in terms of throughput time and overall cost. Concurrently, Li et al. (2017) formulate a 0–1 linear programming model to minimize the total time required for order assignment and rack scheduling, focusing on single picking station scenarios. Building on this, Wang et al. (2022) expand the model to multiple picking stations and acknowledge the critical influence of human workers' fluctuating working states (due to factors like fatigue and circadian rhythms) on warehouse operations. This is a significant departure from typical scheduling models that treat human productivity as a static measure. Further, by introducing the concept of inter-picking station operations, a two-stage hybrid heuristic algorithm is proposed to simultaneously optimize more aspects of warehouse operations, including order assignment, order sequencing, rack selection, and rack sequencing (Wang et al., 2023). Moreover, Zhen et al. (2023) introduce a detailed and elegant model that integrates the scheduling of orders, robots, and storage pods within RMFS, and the order fulfillment time is minimized by efficiently scheduling all components in unison, addressing the complex interactions between them. In summary, performance time serves as a direct indicator of efficiency, rendering it the most compelling metric for the objective function. Consequently, we opt for makespan as the objective function in our subsequent MIP model.

2.3. Tote-based parts-to-picker systems

Most research employs queuing network methods for analyzing the performance of tote-based parts-to-pick systems. Roy et al. (2012) utilize a semi-open queuing network and investigated the impact of the location and assignment of AGVs as well as the number of storage areas on the performance of single/double-deep AVS/RSs. Contrasting with the traditional sequential processing, Zou et al. (2016) introduce a parallel processing policy for tier-captive AVS/RS, enabling the lift and the vehicle to operate simultaneously. And they develop a fork-join queuing network model to analyze the system's performance. Subsequently, they investigate the operational strategies in robotic compact storage and retrieval system (RCSRS) and develop semi-open queuing networks for performance evaluation, which makes significant contributions to the field of RCSRS (Zou et al., 2018a). Besides, Tappia et al. (2017) develop a multi-class queuing network for single-tier and multi-tier compact AVS/RS, finding that setting the depth-to-width ratio around 1.25 in single-tier system minimizes expected throughput time.

Numerous studies have also examined the scheduling challenges in tote-based systems. Some researchers employ methods based on reinforcement learning. Particularly, (Ekren and Arslan, 2022) present an innovative design for an SBS/RS that allows shuttles to move between system tiers. They utilize the Q-learning algorithm for transaction selection, proving its effectiveness over static scheduling algorithms. Later, they propose a Deep Q-Learning approach, poised for broad application in tier-to-tier SBS/RS, by combining it with first-come-first-serve and shortest process time methods (Arslan and Ekren, 2023). Comparatively, other researchers have concentrated on operational optimization framework. Yang et al. (2023) introduce a two-stage heuristic to explore the scheduling of shuttle movements and pickup requests within shuttle-based deep-lane storage systems to minimize makespan. During this time, focusing on SBS/RS featuring two independent lifts shared within a single mast, Chen et al. (2023) propose a decomposition-based heuristic to address sequencing and assigning retrieval requests, aiming to minimize makespan.

Within parts-to-picker systems, additional research has concentrated on the AMR-assisted system. The picker engages in manual picking within the storage area, while AMRs shuttle between the handover location and the depot. Löffler et al. (2022) propose a dynamic routing algorithm for the AMR-assisted system, which is an extension of the method of Ratliff and Rosenthal (1983), achieves an exact solution in polynomial time. Žulj et al. (2022) also investigate the human–robot collaborative order picking assisted by AMRs, aiming to minimize total tardiness for all orders, where an MIP model is established and heuristics are applied.

Accordingly, prior research has investigated both manual and robotic warehousing systems, including SBS/RS and RMFS. These studies have concentrated on single or partial operational optimization challenges, such as order sequencing, tote sequencing, and robot scheduling. Regarding the MTSR AMR system, only one study exists, wherein Qin et al. (2024) develop a queuing network model to analyze the throughput time and capacity under various scenarios. However, based on the current review papers, the joint optimization in order fulfillment of the MTSR AMR system has not been previously investigated (Azadeh et al., 2019; Boysen et al.,

Table 1
Summary of studies on the order fulfillment optimization in various warehousing systems.

	Papers	Objective	Assignment decision			Sequencing decision			Scheduling decision		Routing decision		Multi-station	Solution method
			Orders	Pods	Totes	Orders	Pods	Totes	Robots	Human	Robots	Human		
Manual system	Muter and Öncan (2015)	Travel distance	✓									✓		C&G
	Menéndez et al. (2017)	Total tardiness	✓			✓						✓		Heuristic
	Matusiak et al. (2017)	Service time	✓			✓						✓		Heuristic
	Žulj et al. (2018)	Travel distance	✓									✓		Heuristic
	Ardjmand et al. (2018)	Makespan	✓									✓		Heuristic
	Muter and Öncan (2022)	Makespan	✓							✓		✓		C&G
SBS/RS	Ekren and Arslan (2022)	Cycle time							✓					RL
	Arslan and Ekren (2023)	Process time							✓					RL & Heuristic
	Yang et al. (2023)	Makespan						✓	✓		✓			Heuristic
	Chen et al. (2023)	Makespan			✓			✓	✓					Heuristic
AMR-assisted system	Löffler et al. (2022)	Travel distance				✓					✓			Exact & Heuristic
	Žulj et al. (2022)	Total tardiness	✓		✓								✓	Heuristic
RMFS	Li et al. (2017)	Total time	✓	✓		✓								Heuristic
	Boysen et al. (2017)	Pod visits		✓			✓							Heuristic
	Xiang et al. (2018)	Pod visits	✓	✓										Heuristic
	Jiang et al. (2020)	Pod visits		✓									✓	Heuristic
	Yang et al. (2021)	Pod visits		✓			✓							Heuristic
	Valle and Beasley (2021)	Pod visits	✓	✓			✓						✓	Heuristic
	Xie et al. (2021)	Pod visits	✓	✓									✓	Heuristic
	Wang et al. (2022)	Makespan		✓									✓	ADP-B&P
	Zhuang et al. (2022)	Pod visits	✓	✓		✓	✓		✓				✓	Heuristic
	Wang et al. (2023)	Makespan	✓	✓			✓						✓	Heuristic
	Zhen et al. (2023)	Makespan	✓	✓			✓				✓		✓	Heuristic
MTSR	Our paper	Makespan	✓		✓	✓		✓	✓		✓		✓	Heuristic

2023). This paper addresses order fulfillment in the MTSR AMR system, with a primary objective of minimizing the makespan. It conducts joint optimization of order assignment and sequencing, tote assignment and sequencing, as well as robot scheduling and routing (shown in Table 1). This study expands the research scope of the order fulfillment optimization problem of parts-to-picker systems.

3. Multi-workstation order and tote sequencing and robot scheduling problem

3.1. Problem description

The study explores multi-workstation order and tote sequencing and robot scheduling problems (MOTSRSP) within an MTSR AMR system. Fig. 2 illustrates the layout of a dual-block rectangular MTSR AMR system. In this system, all items are meticulously stored in totes, which conform to uniform specifications and sizes. These totes are positioned on racks that line both the left and right sides of the vertical aisles. There are three cross aisles in the warehouse. All workstations are centrally located at the front cross aisle of the warehouse.

Based on the actual warehouse operations, we provide a detailed explanation of the critical components of MOTSRSP, outlined as follows:

- Order arrival: Orders arrive continuously. Order fulfillment is completed in waves. After completing the order fulfillment for the current wave, the succeeding wave is initiated, ensuring a continuous and dynamic fulfillment of orders.
- Put wall: Each workstation is equipped with a compact shelf, known as a put wall, for storing order boxes. When an order is assigned to a workstation, the corresponding order box is placed on the put wall. Robots are tasked with transporting the designated totes from the storage area to the appropriate workstations. The picker at the workstation, in response to the demands of the orders at the put wall, selects the specified items from the transported tote into the corresponding order box. A workstation can simultaneously hold up to C_w order boxes on its put wall. Upon picking all required items for an order on the put wall, the completed order departs from the workstation, consequently freeing up an empty position on the put wall. A new order then occupies this position. Note that once a tote fulfills the requirements of the put wall, it promptly leaves the workstation. Therefore, the orders that subsequently enter the workstation cannot be fulfilled by the totes that arrived earlier and have already left.
- Tote set: A tote set, denoted as b, consists of multiple totes transported in a single tour by the robot. The quantity of totes in each tote set is constrained by C_r, representing the number of buffer positions of the robot, while fulfilling all orders necessitates multiple tote sets. Each tote set exclusively caters to the order requirements of a single workstation.
- Tote set handling task: The robot handles and transports the assigned tote sets one by one, a process referred to as the tote set handling task. Upon receiving a tote set handling task, the robot embarks on a tour to complete the assignment. The robot starts this tour from the workstation, travels to the storage area to retrieve and transport the tote back to the workstation, unloads the tote on the conveyor, and then departs to execute the ensuing task. The time required to complete each tote set handling

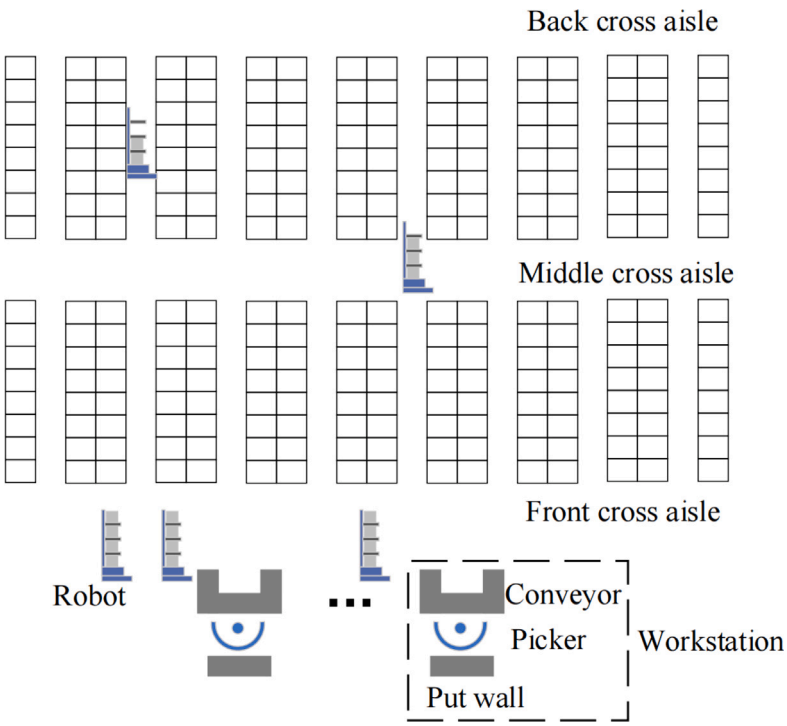


Fig. 2. The layout of MTSR AMR System.

task comprises: (1) travel time, which is the duration the robot takes to leave the workstation, visit each tote's location on the racks, and return to the workstation; (2) pickup time, denoting the period the robot spends picking totes at each shelf location; (3) waiting time, the waiting period if another robot is present and unloading totes at the target workstation; (4) setup time, the interval for the robot to enter the workstation from the waiting area and initiate the unloading process; and (5) unloading time, necessary for unloading the totes onto the conveyor at the workstation.

- Time dimension: Different from the time slot modeling approach (Zhuang et al., 2022), which models different workstations individually in the time slots, we define the time dimension by the sequence in which the tote sets arrive at the workstation and the sequence in which the robot moves the tote sets. In this context, $z_{i,s,k,b}$ denotes that item s in order i is fulfilled by tote set b at workstation k. Simultaneously, we integrate order and tote sequencing with robot scheduling by determining the arrival sequence of the tote set at each workstation $\gamma_{k,b,d}$ and the handling sequence of the robot handling the tote set $\delta_{b,d}$. The variable $\theta_{b,s}$ signifies the transportation status of tote s within the tote set b.
- Makespan: This study minimizes the makespan, which is defined as the duration to complete all orders in the order wave. The completion of the order wave is acknowledged when all totes containing the necessary items for the order wave are transported, subsequently unloaded, and picked at the workstation.

Efficient order fulfillment in MTSR AMR systems hinges on the critical decisions outlined below: (1) Assigning orders to multiple workstations as well as determining the order sequence; (2) Identifying the appropriate tote set for transportation to the workstation, along with the movement sequence of the totes; (3) Scheduling the robots to transfer the required tote to the designated workstation. Fig. 3 illustrates an example to clarify the order fulfillment process.

In this scenario, there are three robots and two workstations, and each robot is equipped with four buffer positions. The capacity of the put wall at each workstation is three order boxes, allowing each workstation to process up to three orders at a time simultaneously. The capital letters in the figure represent the item requirements of this order. For example, the requirements for order O_2 include item A and item B. Initially, the MTSR AMR system assigns the order waves to two workstations. In Fig. 3, the orders assigned to workstation 1 include $O_2 = \{A, B\}$, $O_5 = \{C, D, H\}$, $O_7 = \{D, F\}$, $O_{11} = \{E, G, K\}$, $O_8 = \{I, J, K, L\}$ and $O_3 = \{J, L\}$ with a specified sorting sequence $[O_2, O_5, O_7, O_{11}, O_8, O_3]$. Simultaneously, the system selects the suitable totes in the storage area to fill orders and assigns these totes to the corresponding workstations. For workstation 1, the generated tote sets are $[A, B, C, D]$, $[E, F, G, H]$ and $[I, J, K, L]$ in sequence, and based on this, three robots are scheduled to transport the required totes.

As workstation 1 can accommodate up to three order boxes concurrently, following the order sequence, the initial placement involves O_2, O_5, and O_7. Synchronously, the blue robot transports totes A, B, C and D, completing the picking for O_2. Once fulfilled, O_2 leaves the put wall, making an empty position for O_{11}, which is then placed for picking, and O_5 and O_7 remain on the put wall awaiting completion. Subsequently, the black robot transports totes E, F, G and H, fulfilling orders O_5 and O_7. As a result, O_8 and

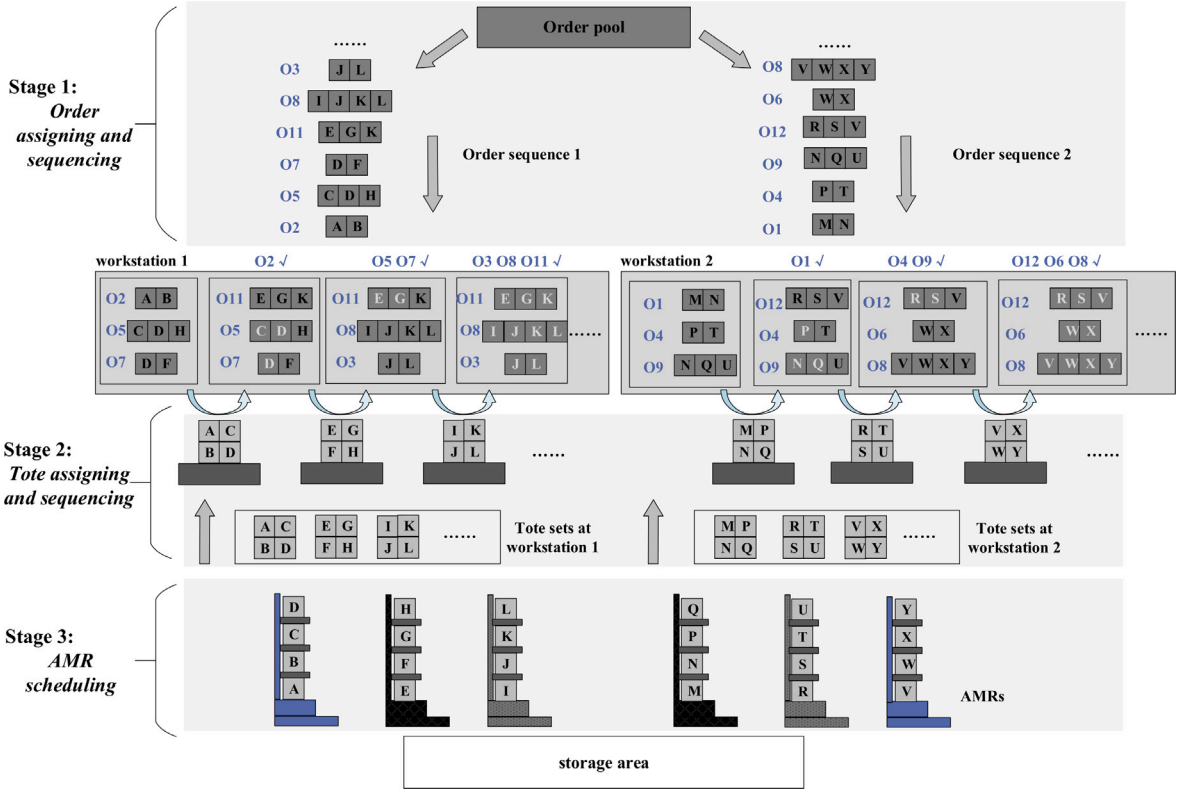


Fig. 3. The MOTSRSP in MTSR AMR System.

O_3 are introduced to the put wall, while O_{11} remains unfinished. In the following step, the gray robot transports incoming totes I, J, K, and L, fulfilling items for all O_{11}, O_8, and O_3. At this point, all six orders assigned to workstation 1 are fulfilled. A parallel process is executed at workstation 2.

There are several assumptions in MOTSRSP:

- Each tote is designated to contain only a single type of item, with each tote being linked to a distinct storage location within the system. The number of the items in the totes being transported is always sufficient. The warehouse encompasses S storage locations, aligning with the item types available.
- At the commencement of the order fulfillment, a total of R identical robots are stationed at the workstation.
- Tote return is not considered in MOTSRSP, as it is managed by a separate robot fleet based on observations of real-world scenarios. This approach avoids the complexity of handling both retrieval and return tasks simultaneously, thereby expediting order fulfillment during the peak order periods.

3.2. Mathematical model formulation

The order sequencing directly influences the sequencing of totes, which in turn is crucial for the efficient scheduling of robots to complete their transport tasks. The decision made in the preceding stage significantly impacts the solution space for the subsequent stage. Due to the complexity of the issue, we propose an integrated approach that encapsulates these three stages of decision-making. We develop the MIP model to formulate the MOTSRSP. In this model, the correlations between the sequencing of orders and totes and the scheduling of robots are accounted for to minimize the overall makespan. Table 2 enumerates all the parameters and variables incorporated in the MIP model.

The MIP model is aimed at minimizing the makespan of the fulfillment of all orders.

$$\min C_{max} \tag{1}$$

- **Constraints of order assignment and sequencing:**

$$\sum_{b\in B}\sum_{k=1}^{K} z_{i,s,k,b} \geq O_{i,s}, \qquad \forall i \in \{1,\dots,N\}, s \in \{1,\dots,S\}, \tag{2}$$

$$x_{i,k,b} \geq z_{i,s,k,b}, \quad \forall i \in \{1,\dots,N\}, s \in \{1,\dots,S\}, k \in \{1,\dots,K\}, b \in B, \tag{3}$$

$$\sum_{b \in B} x_{i,k,b} \leq w_{i,k}|B|, \quad \forall i \in \{1,\dots,N\}, k \in \{1,\dots,K\}, \tag{4}$$

$$\sum_{k=1}^{K} w_{i,k} = 1, \quad \forall i \in \{1,\dots,N\}, \tag{5}$$

$$\sum_{i=1}^{N} x_{i,k,b} \leq C_w, \quad \forall k \in \{1,\dots,K\}, b \in B, \tag{6}$$

$$x_{i,k,b} + x_{i,k,b''} \leq (1 - \gamma_{k,b,b'}\gamma_{k,b',b''} + 1 + x_{i,k,b'}), \quad \forall i \in \{1,\dots,N\}, k \in \{1,\dots,K\}, b, b', b'' \in B, \tag{7}$$

$$\sum_{i=1}^{N} w_{i,k} \leq \frac{N}{K}(1+\tau), \quad \forall k \in \{1,\dots,K\}, \tag{8}$$

Constraints (2) state the fulfillment of all items in the orders. The number of tote sets $|B|$ is a pre-set value.The value of $|B|$ should not be too small, which will result in an infeasible solution. Conversely, a larger value will increase the solving time. When estimating $|B|$, we assume that the initial orders placed into the workstations are processed in batches. New order batches are added to the workstation in batches only after the item requirements of all orders in the current batch are fulfilled. This batch processing mode requires more tote transport tours compared to the sequential order processing mode in our model, where a new order is added to the workstation as soon as an order is completed (Boysen et al., 2023). Therefore, determining $|B|$ using the batch processing mode ensures that the number of tote sets is sufficient to meet all order demands while preventing $|B|$ from being too large. Given N orders are arranged in ascending order by their sequence numbers and distributed to the workstations. Subject to put wall capacity limitations, a total of $\left\lceil \frac{N}{C_w} \right\rceil$ order batches are generated, where NI_c represents the number of item types required by all orders in the order batch c. Subject to robot capacity limitations, the number of tote sets needed to fulfill batch c is $\left\lceil \frac{NI_c}{C_r} \right\rceil$. Therefore, the formula for calculating $|B|$ is as follows:

$$|B| = \sum_{c=1}^{\left\lceil \frac{N}{C_w} \right\rceil} \left\lceil \frac{NI_c}{C_r} \right\rceil$$

Constraints (3) ensure the fulfillment of each order. Constraints (4) dictate that the order is fulfilled at workstation k. Constraints (5) ensure that an order can only be completed on a single workstation. Constraints (6) represent that during the processing of tote set b at workstation k, the count of order boxes on the put wall is less than C_w. Constraints (7) make sure that when an order is placed on the put wall, it cannot leave until the picking is complete; that is, it cannot re-enter once leaving. Constraints (8) limit the load balancing of different workstations. The parameter τ serves to regulate the balance of order distribution among various workstations. As τ decreases, a more even distribution of orders across workstations becomes necessary. Specifically, when τ is set to 0, it implies that the N orders should be distributed uniformly across the K workstations.

- **Constraints for tote assignment and sequencing:**

$$y_{b,k}\theta_{b,s} \geq z_{i,s,k,b}, \quad \forall i \in \{1,\dots,N\}, s \in \{1,\dots,S\}, k \in \{1,\dots,K\}, b \in B, \tag{9}$$

$$\sum_{s=1}^{S} \theta_{b,s} \leq C_r, \quad \forall b \in B, \tag{10}$$

$$\gamma_{k,b,d} + \gamma_{k,d,b} = y_{b,k} * y_{d,k}, \quad \forall b, d \in B, b \neq d, k \in \{1,\dots,K\}, \tag{11}$$

$$\sum_{b \in B} f_b \leq R, \tag{12}$$

$$f_d + \sum_{b \in B, b \neq d} \delta_{b,d} = \sum_{k=1}^{K} y_{d,k}, \quad \forall d \in B, \tag{13}$$

$$\sum_{d \in B, d \neq b} \delta_{b,d} \leq 1, \quad \forall b \in B, \tag{14}$$

Constraints (9) indicate that tote set b is processed at workstation k, and item s is in the tote set b. Constraints (10) determine that the quantity of totes in set b is below the capacity of the robot. Constraints (11) determine the processing sequence of tote set b and tote set d at the workstation k. These nonlinear constraints can be replaced by the following linear constraints:

$$\gamma_{k,b,d} + \gamma_{k,b,d} \leq y_{b,k}, \quad \forall b, d \in B, b \neq d, k \in \{1,\dots,K\}, \tag{15}$$

$$\gamma_{k,b,d} + \gamma_{k,b,d} \leq y_{d,k}, \quad \forall b, d \in B, b \neq d, k \in \{1,\dots,K\}, \tag{16}$$

$$\gamma_{k,b,d} + \gamma_{k,b,d} \geq y_{b,k} + y_{d,k} - 1, \quad \forall b, d \in B, b \neq d, k \in \{1,\dots,K\}, \tag{17}$$

Constraints (12) indicate that the initial tote sets transported by the robots must not exceed R, which limit the number of tote set handling sequences to no more than the number of robots. Constraints (13) ensure that tote set d is either the robot's initial tote

Table 2
Parameters and variables.

Parameters	
S	The number of different items, $s \in \{1, 2, \dots, S\}$
N	The number of orders, $i \in \{1, 2, \dots, N\}$
R	The number of robots, $j \in \{1, 2, \dots, R\}$
K	The number of workstations, $k \in \{1, 2, \dots, K\}$
$\lvert B \rvert$	The number of tote sets in order fulfillment, $b \in B$
$O_{i,s}$	1, if item s is in order i, 0 otherwise
C_w	The capacity of the put wall at the workstation
C_r	The capacity of the multi-tote robot
t_{unload}	The time robot unloads a tote at the workstation
t_p	The time robot picks a tote in the storage area
$t_{s,h}$	The time robot moves from location point s to point h
t_{set}	The time interval between the robot entering the workstation to unload the tote
τ	The workload balance ratio
Binary variables	
$z_{i,s,k,b}$	1, if item s in order i is delivered by tote set b at workstation k, 0 otherwise
$x_{i,k,b}$	1, if order i is delivered by tote set b at workstation k, 0 otherwise
$w_{i,k}$	1, if order i is delivered at workstation k, 0 otherwise
$y_{b,k}$	1, if tote set b is used at workstation k, 0 otherwise
$\theta_{b,s}$	1, if item s is in tote set b, 0 otherwise
$\gamma_{k,b,d}$	1, if tote set b is processed earlier than tote set d at workstation k, 0 otherwise
f_b	1, if tote set b is the first handling task for the arbitrary robot, 0 otherwise
$\delta_{b,d}$	1, if tote set b is delivered directly before tote set d, 0 otherwise
$u_{b,s,h}$	1, if item s is delivered before item h when processing tote set b, 0 otherwise
Continuous variables	
$a_{b,k}$	The arrival time of tote set b at workstation k
$e_{b,k}$	The entry time of tote set b at workstation k
$l_{b,k}$	The departure time of the robot responsible for transport tote set b at workstation k
$T^p_{b,s}$	The picking time of item s in tote set b
$v_{b,s}$	The picking sequence of item s in tote set b, to eliminate the sub-tour
C_{max}	Makespan

set or a direct successor to another tote set b, specifically constraining the utilized tote set d. Constraints (14) ensure the existence of at most one direct successor to tote set b in the handling sequence. Constraints (12), (13), and (14) obtain up to R different tote set handling sequences. Since all robots are identical and the objective function is to find the makespan of all handling tasks completed, the tote set handling sequence obtained by the model can be arbitrarily assigned to different robots. Moreover, each tote set handling task is matched with a robot in sequence.

- **Constraints of the robot routing for each tote set:**

$$\sum_{s \in H, s \neq h} u_{b,s,h} = \theta_{b,h}, \qquad \forall h \in \{1, \dots, S\}, \tag{18}$$

$$\sum_{h \in H, h \neq s} u_{b,s,h} = \theta_{b,s}, \qquad \forall s \in \{1, \dots, S\}, \tag{19}$$

$$\sum_{s \in S} u_{b,s,0} = 1, \qquad \forall b \in B, \tag{20}$$

$$\sum_{h \in S} u_{b,0,h} = 1, \qquad \forall b \in B, \tag{21}$$

$$v_{b,h} \geq v_{b,s} + \sum_{s=1}^{S} \theta_{b,s} * (u_{b,s,h} - 1) + 1, \qquad \forall s, h \in \{1, \dots, S\}, \tag{22}$$

Due to the proximity of all the workstations, and considering that the distance between them is negligible compared to the picking distance, we have configured the coordinates of several workstations to be at the same point 0. This setting focuses on the efficiency of the picking process rather than the minor differences in the physical locations of the workstations. Constraints (18)–(21) depict the inflow and outflow balance of each storage location of item sets and the workstation location 0 during the picking of tote set b, where $H = S \cup 0$. Constraints (22) aim to eliminate the sub-tours.

- **Constraints of the time nodes on each tote set b:**

$$T^p_{b,s} + t_{s,0} - M(1 - u_{b,s,0}) \leq a_{b,k}, \qquad \forall s \in \{1, \dots, S\}, k \in \{1, \dots, K\}, b \in B, \tag{23}$$

$$T^p_{b,s} + t_{s,h} + t_p - M(1 - u_{u,b,h}) \leq T^p_{b,h}, \qquad \forall s, h \in \{1, \dots, S\}, b \in B, h \neq s, \tag{24}$$

$$a_{b,k} \le e_{b,k}, \qquad \forall k \in \{1,\dots,K\}, b \in B, \tag{25}$$

$$l_{b,k} - M(1-\gamma_{k,d,b}) \le e_{k,d}, \qquad \forall k \in \{1,\dots,K\}, b,d \in B, \tag{26}$$

$$e_{b,k} + t_{set} + \sum_{s=1}^{S} \theta_{b,s} * t_{upload} \le l_{b,k}, \qquad \forall s \in \{1,\dots,S\}, k \in \{1,\dots,K\}, b \in B, \tag{27}$$

$$l_{b,k} + t_{0,s} + t_p - M(2-\delta_{b,d} - u_{d,0,s}) \le T^p_{d,s}, \qquad \forall s \in \{1,\dots,S\}, k \in \{1,\dots,K\}, b,d \in B, \tag{28}$$

$$t_{0,s} + t_p - M(1-u_{b,0,s}) \le T^p_{b,s}, \qquad \forall s \in \{1,\dots,S\}, k \in \{1,\dots,K\}, b \in B, \tag{29}$$

Constraints (23) restrict the time of the robot loaded tote set b arriving the workstation k, where M is a constant large number. Constraints (24) regulate the picking time of each item h in the tour of tote set b, when the robot moves from point s to point h. Constraints (25) ensure that the robot loaded tote set b can enter workstation k only after arriving it. Constraints (26) indicate that the robot with tote set d can enter only when the other robot for transport tote set b leaves the workstation k. Constraints (27) detail the time from entry to exit of the workstation of the robot. Upon unloading the tote set b, the robot promptly leaves the workstation k without waiting for the picker to finish the item picking. Furthermore, the setup time t_{set} guarantees that once the current robot enters the workstation, the picker has already completed the item picking for the preceding tote sets. Consequently, there is no need to incorporate the picking time of items into the model constraint. Constraints (28) indicates that the robot can only begin to complete the subsequent tote set d handling after the previous tote set b handling is completed. Constraints (29) limit the picking time of the first item s in any tote set b.

- **Constraints for makespan:**

$$l_{b,k} \le C_{max}, \quad \forall k \in \{1,\dots,K\}, b \in B, \tag{30}$$

Constraints (30) ensure that the makespan C_{max} exceeds the time of all tote sets has been transported to workstations.

- **Constraints for decision variables:**

$$z_{i,s,k,b} \in \{0,1\}, \qquad \forall i \in \{1,\dots,N\}, s \in \{1,\dots,S\}, k \in \{1,\dots,K\}, b \in B, \tag{31}$$

$$x_{i,k,b} \in \{0,1\}, \qquad \forall i \in \{1,\dots,N\}, k \in \{1,\dots,K\}, b \in B, \tag{32}$$

$$w_{i,k} \in \{0,1\}, \qquad \forall i \in \{1,\dots,N\}, k \in \{1,\dots,K\}, \tag{33}$$

$$y_{b,k} \in \{0,1\}, \qquad \forall k \in \{1,\dots,K\}, b \in B, \tag{34}$$

$$\theta_{b,s} \in \{0,1\}, \qquad \forall s \in \{1,\dots,S\}, b \in B, \tag{35}$$

$$\gamma_{k,b,d} \in \{0,1\}, \qquad \forall k \in \{1,\dots,K\}, b,d \in B, b \ne d, \tag{36}$$

$$f_b \in \{0,1\}, \qquad \forall b \in B, \tag{37}$$

$$\delta_{b,d} \in \{0,1\}, \qquad \forall b,d \in B, b \ne d \tag{38}$$

$$u_{b,s,h} \in \{0,1\}, \qquad \forall s,h \in \{1,\dots,S\}, b \in B, \tag{39}$$

$$a_{b,k}, e_{b,k}, l_{b,k} \ge 0, \qquad \forall k \in \{1,\dots,K\}, b \in B, \tag{40}$$

$$T^p_{b,s}, v_{b,s} \ge 0, \qquad \forall s \in \{1,\dots,S\}, b \in B, \tag{41}$$

Constraints (31)–(41) represent requirements on the decision variables.

4. Solution approach

MOTSRSP encompasses various decisions related to order and tote sequencing and robot scheduling. When considering solely order and tote sequencing, the challenge of optimizing order fulfillment is classified as NP-hard (Boysen et al., 2023). Additionally, the routing within the robot scheduling decision, treated as a vehicle routing problem, is also NP-hard. Given the complexity of the problem, we proffer a tailored I-ALNS algorithm to solve the MOTSRSP. A two-stage heuristic algorithm is developed as a component of I-ALNS to address the challenges of order and tote sequencing, as well as robot scheduling, respectively. This I-ALNS exhibits efficacy in swiftly resolving intricate large-scale instances and can find optimal solutions in most small-scale scenarios with a short calculation time.

The flow chart of the I-ALNS algorithm is shown in Fig. 4. We initially present an item similarity and popularity-driven method (ISPDM) to acquire order and tote sequencing at the workstations (Section 4.1) in the first stage. Subsequently, in the second stage (Section 4.2), we employ a genetic algorithm (GA) to schedule robots, directing them to manipulate all totes following the predetermined tote sequencing. The maximum completion time, indicating the makespan of order fulfillment is determined as the robot transports all the totes to workstations. I-ALNS involves the destroy and repair operations of the order sequence acquired in ISPDM (Section 4.3), yielding a novel order sequence. Conforming to this new order sequence, we utilize ISPDM with the given order sequence to obtain the tote sequence (Section 4.1.3). Ultimately, the GA algorithm is deployed to ascertain the makespan for the completion of all tote-handling tasks by the robot. The iteration of I-ALNS continues until the termination condition is achieved.

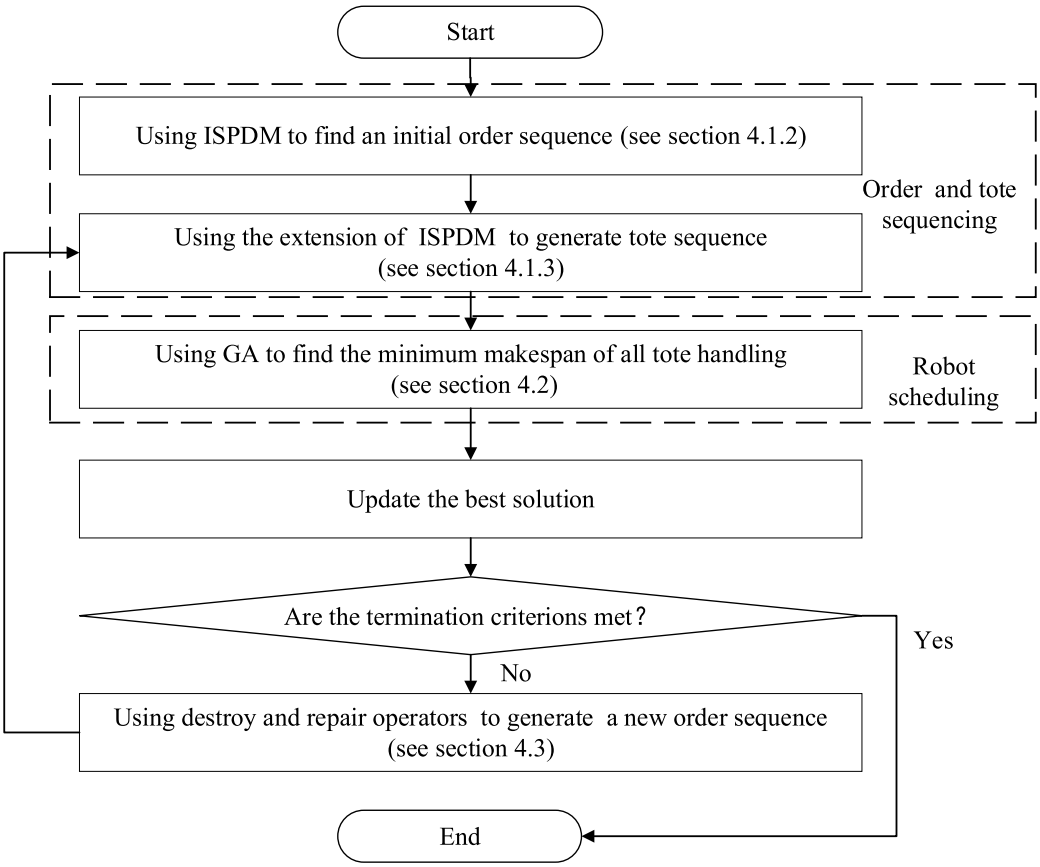


Fig. 4. The flow chart of our I-ALNS.

4.1. Order and tote sequencing problem

We present the ISPDM, which can quickly get the solutions for order and tote sequencing problems. In section 4.11, we first introduce some basic settings and order sequencing structures, and 4.12 specifically introduces ISPDM. Section 4.13 extends ISPDM to develop an item popularity-driven method (IPDM) that can generate a corresponding tote sequence with a given order sequence.

4.1.1. Basic setting

First, some basic settings are introduced. We use $O = [O_1, O_2, \dots, O_N]$ to express N order demands, where O_i represents the order i. Each order O_i is a set containing different item requirements. The warehouse comprises K workstations, each with a put wall possessing a capacity of C_w. Define OW to represent the order sequence at the workstations and $OW[k]$ to represent the order sequence at workstation k. Define the tote sequence TW, $TW[k]$ is the tote arrival sequence at the workstation k. For instance, there are two workstations, each with a put wall capacity of two order boxes. The robot is equipped with two buffer positions, and the wave order O comprises four distinct orders, namely O_1, O_2, O_3, O_4, which can be represented as $O_1 = [s_1, s_3]$, $O_2 = [s_2, s_5, s_6]$, $O_3 = [s_1, s_4]$, and $O_4 = [s_5, s_6, s_7]$. A order sequence $OW = [[O_1, O_3], [O_2, O_4]]$, can obtain a feasible tote sequence $TW = [[s_1, s_3, s_4], [s_2, s_5, s_6, s_7]]$.

Intuitively, a reduced number of robot tours tends to decrease the makespan. This idea is derived from a study in RMFS where the number of moves of movable shelves is positively correlated with makespan (Zhuang et al., 2022). Therefore, we let the robot load more totes in one tour as much as possible, and if there are enough types of totes required, the full capacity of the totes will be carried to the workstation. For a robot with 2 buffer positions, the TW can be expressed as $[[[s_1, s_3], [s_4]], [[s_2, s_5], [s_6, s_7]]]$. $TW[k][l]$ (where l shall not exceed the number of tote set in $TW[k]$) can be used to represent the l−th tote set to arrive at workstation k. $TW[1][1]$ is the tote set $[s_1, s_3]$, which is the first tote set to arrive at workstation 1. We define sets COW, CTW, FOW, and FTW, respectively, representing the current order at workstations, the current totes required by workstations, the finished order in workstations, and the used totes of workstations. $COW[k], CTW[k], FOW[k]$, and $FTW[k]$ represent the corresponding meaning at workstation k. The order completion process includes from order pool O to $COW[k]$ to $FOW[k]$, and CTW is generated according to the demand

Table 3
Description of symbols.

Symbols	Description
OW	The order sequence at the workstations
$OW[k]$	The order sequence at workstation k
TW	The tote sequence
$TW[k]$	The tote arrival sequence at the workstation k
COW	The current order at workstations
$COW[k]$	The current order at workstation k
CTW	The current totes required by workstations
$CTW[k]$	The current totes required by workstation k
FOW	The finished order in workstations
$FOW[k]$	The finished order in workstation k
FTW	The used totes of workstations
$FTW[k]$	The used totes of workstation k

of COW, and the used totes are added to FTW. we can use $[O, COW, CTW, FOW, FTW]$ to state all information on the orders and totes sequencing. Table 3 illustrates the description of symbols.

4.1.2. Item similarity and popularity-driven method

Assigning different orders with the same item requirements to an identical workstation allows the robots to fulfill the same item demands of different orders in a single tour. Reducing the number of tours in which the robots transport the totes to the workstations minimizes the makespan. Therefore, minimizing the number of tours for transporting totes is an intuitive and rational scheduling rule. Consequently, we devised the order similarity-driven method to derive an order and tote sequence, minimizing the robot's movements. In the ISPDM, incorporating orders into the workstation emulates a seed algorithm's principles. It calculates the similarity based on the item category with pending orders in the workstation. Orders are then sorted in descending order of similarity and added to the workstation. Simultaneously, the methodology for selecting tote transportation sequences prioritizes totes with higher item quantity demands. This approach enhances operational efficiency by ensuring that each transportation cycle contributes to fulfilling a maximum number of different orders, optimizing resource utilization, and streamlining warehouse workflow.

The overview of the ISPDM is shown in Algorithm 1. Initially, all orders are placed in the order pool, with COW, CTW, FOW, and FTW remaining empty, as outlined in line 1. The iteration begins and proceeds in a loop until the order pool is empty and no orders remain in the workstation (line 2). If the capacity of the put wall at the workstation is not reached and there are still orders in the order pool that, together with completed orders, do not surpass the workload balance (line 4), the addition of orders to the workstation begins. If there are no orders in the workstation (line 5), a seed order O_{sd}, which has the most categories of items in the order pool, is selected and added into the workstation. Subsequently, the similarity between each remaining order in the order pool and the seed order is assessed using the specified formula:

$$Sim_i = \frac{|O_{sd} \cap O_i|}{|O_{sd} \cup O_i|} \tag{42}$$

Sim_i is the similarity between order O_i and seed order. In the formula, the numerator on the right side signifies the shared types of items between the order and the seed order, while the denominator denotes the total distinct types of items in both the order and the seed order. Orders from the order pool are added to $COW[k]$ in descending order of similarity until the conditions outlined in line 4 are no longer satisfied (line 6). When uncompleted orders exist in the workstation (line 7), the unfulfilled items of these orders are combined to form a composite seed order. Subsequently, the similarity between the remaining orders in the order pool and this composite seed order is calculated. Orders are then added to the workstation in descending order of similarity until the condition stipulated in line 4 is no longer met (line 8). Then, we determine the type of totes to be moved according to the order on the put wall. We count all unfulfilled items on the put wall, obtaining the quantity needed for each unfulfilled item in the current put wall. Organizing these quantities in descending order generates a sequence MT_k representing the popularity of totes at workstation k. The first tote in MT_k signifies that the corresponding item is required by the majority of orders concurrently. The top $min(|MT_k|, C_r)$ totes in MT_k are selected and incorporated into $CTW[k]$ (line 11). According to the categories of totes in $CTW[k]$, the corresponding item demands from the orders in $COW[k]$ are removed, indicating that the needs for these items at the put wall have been satisfied. This means that the demand for these items on the put wall has been met. Then $CTW[k]$ is added to $FTW[k]$ as a set, and orders in $COW[k]$ whose all requirements have been fulfilled are moved to $FOW[k]$ (line 12). Upon the completion of the iteration, the resulting FOW and FTW represent the final order sequence OW and the tote sequence TW, respectively.

Algorithm 1 Item similarity and popularity-driven method

Input: order pool $O = \{O_1, O_2, ..., O_n\}$, the number of the workstations K, the number of buffer positions of robot C_r, the capacity of the put wall at the workstation C_w, workload balance ratio τ

Output: order sequence $\boldsymbol{OW}$,tote sequence $\boldsymbol{TW}$

1: **Initialize** current orders in the workstations $COW = \emptyset$,current totes needed for the workstations $CTW = \emptyset$,finished orders $FOW = \emptyset$, finished totes $FTW = \emptyset$
2: **while** $O \neq \emptyset$ or $COW \neq \emptyset$ **do**
3: **for** $k = 1, k \leq K, k++$ **do**
4: **while** $COW[k] \neq C_w$ and $O \neq \emptyset$ and $COW[k] + FOW[k] \leq \frac{N}{K}(1+\tau)$ **do**
5: **if** $COW[k] = \emptyset$ **then**
6: select the order with the most item types from O and add to $COW[k]$ as seed order, calculate the similarity value Sim_i of all remaining orders in O and add the orders with the greatest similarity value to $COW[k]$ in turn
7: **else**
8: set the remaining orders in $COW[k]$ as a seed order, calculate the similarity value Sim_i of the remaining orders in O and add the orders with the greatest similarity value to $COW[k]$ in turn
9: **end if**
10: **end while**
11: statistics MT_k, select top $min(|MT_k|, C_r)$ in the MT_k, join the CTW[k].
12: according to $CTW[k]$, remove the corresponding items in $COW[k]$. Add $CTW[k]$ to $FTW[k]$, and remove finished orders in $COW[k]$ to $FOW[k]$
13: **end for**
14: **end while**
15: $OW \leftarrow FOW$, $TW \leftarrow FTW$
16: **return** OW, TW

4.1.3. Tote sequencing with a given order sequence

This section expands the ISPDM to generate a tote sequence based on a given order sequence. Unlike the original process, this adaptation does not require evaluating similarities to determine the order sequence. This extension of ISPDM, termed the item popularity-driven method (IPDM), becomes an integral element of our designed I-ALNS algorithm. Utilizing the new order sequences resulting from destroy and repair operations as input, IPDM efficiently produces a corresponding tote sequence.

In the initial phase, all order sequences OW are determined, and sets COW, CTW, FOW, and FTW are empty (line 1). The iteration begins and continues in a loop until there are no orders in OW, and the workstation COW has no orders remaining (line 2). If the capacity of the put wall at the workstation is not reached and the aggregate of existing and completed orders does not exceed the workstation's load capacity (line 4), the process of integrating orders into the workstation commences. This involves removing the first order from the sequence $OW[k]$ and adding it to workstation K, repeating this operation until the condition in line 4 is no longer met. Following this, the popularity of all item requirements within the workstation is calculated, and they are ranked in descending order to establish a sequence MT_k. The top $min(|MT_k|, C_r)$ totes in MT_k are selected and added to $CTW[k]$ (line 7). Based on the tote categories in $CTW[k]$, the corresponding item requirements are eliminated from the orders in $COW[k]$. $CTW[k]$ is then added to $FTW[k]$ as a set, and orders in $COW[k]$ whose item requirements have been satisfied, are moved to $FOW[k]$ (line 8). Upon the completion of the iteration, the resulting FTW represents the final tote sequence TW.

Algorithm 2 Item popularity-driven method

Input: order sequence OW, the number of the workstations K, the number of buffer positions of robot C_r, the capacity of the put wall at the workstation C_w, workload balance ratio τ

Output: tote sequence $\boldsymbol{TW}$

1: **Initialize** current orders in the workstations $COW = \emptyset$,current totes needed for the workstations $CTW = \emptyset$,finished orders $FOW = \emptyset$, finished totes $FTW = \emptyset$
2: **while** $OW \neq \emptyset$ or $COW \neq \emptyset$ **do**
3: **for** $k = 1, k \leq K, k++$ **do**
4: **while** $COW[k] \neq c_W$ and $OW[k] \neq \emptyset$ and $COW[k] + FOW[k] \leq \frac{N}{K}(1+\tau)$ **do**
5: select first order in $OW[K]$,and add it to $COW[k]$
6: **end while**
7: statistics MT_k, select top $min(|MT_k|, C_r)$ in the MT_k, join the $CTW[k]$.
8: according to $CTW[k]$, remove the corresponding items in $COW[k]$. Add $CTW[k]$ to $FTW[k]$, and remove finished orders in $COW[k]$ to $FOW[k]$
9: **end for**
10: **end while**
11: $TW \leftarrow FTW$
12: **return** TW

4.2. Genetic algorithm for robot scheduling problem

By implementing the ISPDM or IPDM, it becomes feasible to determine the tote sequences TW for all workstations, wherein each sequence $TW[k]$ at workstation k encompasses a multitude of tote set handling tasks. All the tote task sets are assigned to different robots for transportation, presenting a challenge in robot scheduling. We transform the robot scheduling problem (RSP) into one akin to the flexible job shop scheduling problem (FJSP). In this analogy, all tote set transportation tasks at each workstation are treated as 'jobs' in FJSP, with each tote set task equating to an 'operation' within the job. The robots serve as processing machines, where each robot can only transport another tote set after completing the current one. Since all robots are identical and any tote set can be transported by any robot, with equal transportation time regardless of the robot, the RSP is fully flexible.

GA is one of the most classic methods for solving FJSP. We deploy the effective GA, proposed for RSP, adopting the initialization, crossover, mutation, and selection procedures used in FJSP. Further, a comprehensive description of these procedures is provided in Zhang et al. (2011). However, due to two differences between RSP and FJSP, decoding in FJSP is unsuitable for RSP. Firstly, each operation within a job can commence only following the completion of its preceding operation in FJSP. Conversely, in RSP, subsequent tote sets at the same workstation may be transported by other robots even if a preceding tote set task remains incomplete. Nevertheless, the entry of subsequent tote sets into the workstation must occur after the previous ones. Another difference from FJSP is the variability in processing times within RSP. In FJSP, the processing time for each operation on a machine is predetermined and constant. Conversely, in RSP, the time a robot takes to complete each tote set is variable. In Section 3.1, we elucidate the tote set handling task and demonstrate that the fulfillment time for each task encompasses five distinct components. Notably, the waiting time at the workstation varies depending on different scheduling approaches. We consider this fluctuating and uncertain waiting time to provide a more accurate and realistic representation of the process. Consequently, we devise a specialized decoding scheme tailored for RSP. Details of the encoding and decoding in GA are available in the Appendix.

4.3. Item characteristic-driven adaptive large neighborhood search

We introduce the I-ALNS that incorporates the previous ISPDM and GA as components, which can modify the order sequence to enhance the quality of the solution. The original concept of ALNS is proposed by Røpke and Pisinger (2006). The pseudo-code for the I-ALNS is presented in Algorithm 3. Initially, we utilize the previously discussed ISPDM to obtain an initial order sequence OW, and a tote sequence TW (line 1). GA then determines the minimum makespan for the current tote sequence, denoted as fit (line 2). Subsequently, we set fit as the historical best fitness fit^*, and initialize the weights of the destroy and repair operators, w_i^- and w_i^+, respectively (line 3). In the beginning, all operators are assigned equal weights. Then iterate until the termination condition is satisfied. In this study, the maximum number of iterations does not exceed 100. We first determine the quantity, q^-, of customer orders to be removed (line 5), and choose the destroy and repair operators, op_i^- and op_i^+, based on the probabilities π_i^- and π_i^+ (line 6). Line 7 generates a new order sequence OW' by the destroy and repair operators. Then we obtain TW' using IPDM with the given OW' (line 8), which is subsequently inputted into the GA to determine fit' (line 9). If fit' meets the simulated annealing acceptance criteria, the current order sequence OW and the current fit are updated (line 11). If fit' is less the historical best fitness, the historical best solution is updated on line 14. Next, we adjust the weights of the selected destroy and repair operators, w_i^- and w_i^+, as well as the operator selection probabilities, π_i^- and π_i^+, based on our scoring system (line 15). The algorithm returns the historically best fitness and the solution s^* (line 17). In the subsequent sections, more detailed information about the components of I-ALNS is provided.

Algorithm 3 Item characteristic-driven adaptive large neighborhood search

1: **initialize** generate initial order and tote sequence solution OW, TW by ISPDM
2: calculate initial makespan value fit by GA
3: $fit^* \leftarrow fit, we_i^- = (1, ..., 1), we_i^+ = (1, ..., 1)$
4: **while** $iterations < max_iterations$ **do**
5: determine q^-
6: choose destroy and repair operators (op_i^-, op_i^+) according to probabilities (π_i^-, π_i^+)
7: $OW' \leftarrow op_i^+(op_i^-(OW))$
8: generate tote sequence solution TW' by IPDM
9: calculate makespan value fit' by GA
10: **if** accept SA(fit') **then**
11: $OW \leftarrow OW', fit \leftarrow fit'$
12: **end if**
13: **if** $fit' < fit^*$ **then**
14: $fit^* \leftarrow fit'$
15: **end if**
16: adjust we_i^-, we_i^+, π_i^-, and π_i^+
17: **end while**
18: **return** fit^*

4.3.1. Destroy and repair operators

During each iteration of the I-ALNS algorithm, a new order sequence is generated by applying a destroy and a repair operation. Tailoring to the specific characteristics of our problem, we devise three distinct types of destroy and repair operations.

The number of orders to be destroyed q^- and repaired q^+ in each iteration fluctuates randomly between the interval $[0.2|O|, 0.4|O|]$, where $|O|$ is the total number of wave orders. The order sequence OW encompasses K order sequences at various workstations. During each iteration, N_d order sequences randomly chosen from different workstations undergo destruction and repair operations. For each destructive operation, N_d is randomly generated between 1 to K. The quantity of orders to be destroyed for each sequence is represented as DN.

$$DN = \begin{cases} \lfloor q^- / N_d \rfloor, & \text{the first } N_d - 1 \text{ selected order sequences} \\ q^- - (N_d - 1) * \lfloor q^- / N_d \rfloor, & \text{the last selected order sequence} \end{cases} \tag{43}$$

For the first $N_d - 1$ selected order sequences, $\lfloor q^- / N_d \rfloor$ orders are designated for destruction. The number of orders destroyed in the last sequence equals the total number of orders that need to be destroyed minus the orders already destroyed in other sequences.

Random destroy To add diversity to the order sequences, the selected N_d order sequences are sequentially subjected to a random destroy process. The process for destroying each order sequence is as follows: a random position within the order sequence is chosen, and from this point, a total of DN orders, including the selected order and the subsequent ones, are removed. This process is repeated for each of the N_d chosen order sequences. Additionally, the starting point of destruction for each affected order sequence is recorded.

Fewest destroy Considering the varying characteristics of order categories, we observe that certain items are less frequently demanded across all orders. Orders containing these less-demanded items are infrequent and may not be optimally positioned in the sequence. The main steps of the fewest destroy steps are:

Step a1. Examine all the items in the order pool. Identify the items that are needed least frequently.

Step a2. Then identify the orders that contain these least required items. For each of the selected order sequences earmarked for destruction, if they contain at least one of these identified orders, go to step a3, otherwise go to step a4.

Step a3. One of these orders is randomly selected, and a total of DN orders are removed, starting from the selected order and including subsequent ones.

Step a4. The last DN orders of the sequence are removed.

Additionally, the starting point of destruction for each affected order sequence is recorded.

Worst destroy Before implementing worst destroy, we introduce a metric called the single robot completion time T_{ct}. T_{ct} is the completion time for a robot to complete all the tote-handling tasks in the corresponding tote sequence at one workstation. Where the number of robots is limited, it is observed that the greater T_{ct}, the longer the makespan tends to be. This method specifically targets and removes those orders that contribute the most to the T_{ct}, effectively aiming to minimize the makespan. By eliminating and repositioning these time-intensive orders, we can significantly streamline the processing sequence, thereby enhancing the efficiency of the system. The destruction process is applied sequentially to the selected N_d order sequences. The destruction rule for each order sequence is as follows:

Step b1. Compute T_{ct} for the order sequence generated by deleting the DN orders preceding the selected sequence.

Step b2. Subsequently, calculate T_{ct} for sequences obtained by removing DN orders starting from the second order up to the sequence where the last DN orders are removed.

Step b3. The removal scenario that results in the lowest T_{ct} is selected as the final removal plan.

The starting point of destruction for each affected order sequence is recorded. This systematic approach facilitates the efficient exclusion of order sequences, giving priority to scenarios that minimize T_{ct}.

Since the destruction operation results in N_d destroyed order sequences and q^- removed orders, we develop three distinct repair operations to reintegrate the removed orders into the N_d destroyed order sequences. The number of orders in each repaired order sequence must adhere to the principle of workload balance at the workstation.

Random repair The total of q^- orders removed during the destruction phase are randomly inserted at the points of destruction in each of the destroyed order sequences. This process continues until all the removed orders have been successfully reintegrated into order sequences.

ISPDM repair In this repair process, we employ the ISPDM introduced in Section 4.1.2. This method is applied to the orders removed from the destruction phase, which are initially set as a new order pool. The number of destroyed order sequences, N_d is treated as the number of workstations. This repair rule is as follows:

Step c1. Using ISPDM, we organize N_d order sequences based on the removed orders.

Step c2. The new N_d order sequences obtained through ISPDM are then randomly inserted back into each previously destroyed order sequence.

Greed repair The repair uses the greedy rule to add the removed orders one by one to the different destroyed order sequences. The steps are as follows:

Step d1. Randomly starting with one of the destroyed order sequences, a removed order is selected and added back at the disruption point. The selection criterion for inserting an order into the order sequence is based on minimizing the single robot completion time T_{ct}, for the newly formed order sequence.

Step d2. Step d1 is repeated until all removed orders have been integrated into the order sequences.

4.3.2. Adaptive mechanism

The adaptive weight adjustment in the I-ALNS algorithm dynamically assesses the significance of each destroy and repair operation throughout the search process. The weight assigned to each operation is adjusted based on its performance in previous iterations. Initially, all operations are given equal weight, $we = 1.0$. The selection probability of an operation π is calculated as the weight of that operation divided by the total of all operations' weights. An operation's effectiveness is gauged using a scoring system. If a destroy and repair operation yields a historically best value, it earns a score of $Scr_{best} = 1.5$. When the operation produces a solution superior to the current one, it is awarded a score of $Scr_{better} = 1.2$. Operations that do not enhance the current solution but are accepted based on the simulated annealing acceptance criterion receive a score of $Scr_{pass} = 0.8$. Conversely, operations that do not meet the simulated annealing acceptance criterion are given a score of $Scr_{bad} = 0.5$. Following each destroy and repair cycle in I-ALNS, the corresponding weights are updated according to the formula provided in Eq. (44).

$$we_{t+1} = we_t \cdot c + (1 - c) \cdot \frac{Scr}{\beta} \tag{44}$$

we_{t+1} is the updated weight for the tth iteration, we_t is the current weight, and c is a constant that determines the rate at which the weight is updated, set as 0.4. Scr is the score received based on the performance of the operation in this iteration, and β is the number of times the operation has been used in the past iterations.

4.3.3. Acceptance criterion

In our I-ALNS framework, simulated annealing is employed as the acceptance criterion. A new solution is accepted if it is better than the current solution. If the new solution is worse, it is accepted with a certain probability based on the SA acceptance criterion, which plays a pivotal role in the I-ALNS algorithm. The formula for this criterion is as follows:

$$P(Accept) = e^{-(fit' - fit)/T_c} \tag{45}$$

$P(Accept)$ is the probability of accepting the current solution. T_c is the current temperature. The exponential term in the formula dictates the probability of accepting the new solution: the greater the difference in $fit' - fit$, the lower the probability of acceptance. We set the initial temperature T to 100, allowing for a higher probability of accepting worse solutions initially. Each time a deteriorating solution is accepted, the temperature is reduced, employing a cooling constant factor set to 0.97. As the value of the exponential term decreases, so does the probability of accepting inferior solutions. This mechanism allows the algorithm to accept worse solutions during the search process, aiding in avoiding local optimal and facilitating a global search in the solution space. By incorporating this principle of simulated annealing, the I-ALNS is empowered to explore a broader range of solutions, enhancing its capability to find the optimal solution.

5. Numerical experiments

We conduct numerical experiments to demonstrate the effectiveness and accuracy of the I-ALNS algorithm. Heuristics are compared against the commercial solver Gurobi in small-scale scenarios in Section 5.2, and compared with a real operation method of a warehouse in large-scale scenarios in Section 5.3. Section 5.4 investigates sensitivity analysis regarding robot and workstation parameters in the warehouse, yielding several managerial insights. The experiments are performed on a computer with an i7-8565 CPU at 1.80 GHz, which has 8 GB RAM. The MIP model was solved using Python 3.8 with Gurobi 9.5.1, and all heuristic algorithms are implemented in Python 3.8.

5.1. Experiment settings

The experimental data in this study originates from a third-party logistics company in Shenzhen, China, employing the MTSR AMR system primarily for B2C orders. In this warehouse, the robots maintain an average speed of 1.4 m/s, which accounts for acceleration and deceleration during turns, thus reflecting realistic operational dynamics. The width between the aisles is 2.5 m, and the length of each storage location within the aisles is 0.7 m. The time required for a robot to pick up a tote at the storage location is 5.2 s while unloading each tote at a workstation takes 5 s. Moreover, the average setup time, from the robot's arrival at the workstation area to entering the designated workstation for initiating tote unloading, is 8.6 s. A dual-block experimental warehouse is designed according to these specifications. Each block consists of 20 aisles, each containing 30 storage locations. Each side of the aisle accommodates 15 distinct totes, with a cumulative capacity of 1200 unique items. The workstations are centrally located at the front cross aisle of the warehouse, with each station situated 3 meters vertically from the entrance of the aisles. The horizontal aisle width between the two blocks is 2 m.

In conducting the data analysis, samples of orders spanning seven days are randomly selected for statistical analysis from the warehouse's monthly order records. The warehouse mainly serves B2C operations, with 77.6% of orders being single-line, 17.8% double-line, and 4.6% comprising three or more lines, averaging no more than 1.3 lines per order. In our numerical experiments, we set the presence of single-line, double-line, and triple-line orders in our instance groups, randomly generated with probabilities of 75%, 20%, and 5%, respectively.

We provide 3 tiers of experimental instance groups. The small-scale group encompasses scenarios with 4, 5, 6, and 10 orders; the medium-scale group includes configurations of 15, 20, 40, and 50 orders; and the large-scale group involves arrangements with 80, 120, 160, and 200 orders. In total, these groups cover 12 distinct experimental settings, providing a robust experiment for evaluating

Table 4
Parameter configurations for different instance groups.

Instance groups	Number of orders $\|O\|$	Number of workstations K	Number of robots R	Capacity of put walls C_w	Capacity of robots C_r
SI1	4	2	1	2	2
SI2	5	2	2	3	2
SI3	6	2	3	3	3
SI4	10	3	3	4	4
MI1	15	3	4	6	5
MI2	20	3	5	8	6
MI3	40	3	6	6	5
MI4	50	4	6	8	6
LI1	80	4	6	8	6
LI2	120	4	8	10	8
LI3	160	6	6	16	6
LI4	200	8	10	16	8

Table 5
Evaluating different methods for small and medium-scale instances.

Instance	τ	Gurobi		ISPDM/GA			I-ALNS		
		Obj_G	$Time$ (s)	Obj_S	$Time$ (s)	Gap_S (%)	Obj_A	$Time$ (s)	Gap_A (%)
SI1	0	196.61	2.55	207.32	0.05	5.45	196.61	0.37	0.00
	0.2	192.24	1.85	204.75	0.04	6.50	192.24	1.38	0.00
SI2	0	158.90	27.05	172.75	0.07	8.72	158.90	14.24	0.00
	0.2	139.54	31.98	171.90	0.06	23.19	139.54	12.59	0.00
SI3	0	123.06	89.16	165.77	0.08	34.71	125.34	11.27	1.85
	0.2	103.49	67.98	129.27	0.06	24.19	103.48	7.75	0.00
SI4	0	183.69	203.26	223.11	0.07	21.46	184.48	15.40	0.42
	0.2	186.97	368.23	231.83	0.06	23.99	186.97	10.65	0.00
MI1	0	201.84	1800.00	249.20	0.07	23.46	177.83	15.04	−11.89
	0.2	192.76	1800.00	227.91	0.06	18.23	166.41	16.43	−10.76
MI2	0	248.76	1800.00	248.76	0.06	6.78	221.98	16.17	−11.89
	0.2	226.99	1800.00	262.55	0.07	15.66	198.62	16.31	−12.49
MI3	0	N/A	1800.00	398.25	0.09	N/A	316.88	27.70	N/A
	0.2	N/A	1800.00	423.59	0.08	N/A	328.59	26.99	N/A
MI4	0	N/A	1800.00	485.69	0.08	N/A	355.68	32.55	N/A
	0.2	N/A	1800.00	473.26	0.09	N/A	348.37	31.50	N/A

Note. $Gap_S = (Obj_S - Obj_G)/Obj_G * 100\%$, $Gap_A = (Obj_A - Obj_G)/Obj_G * 100\%$.

our algorithms across varied operational scales. Each instance group encompasses different parameters such as the number of orders n, the number of workstations k, the number of robots R, the capacity of the put wall at the workstation C_w, and the tote capacity of the robots C_r, with each group's parameters detailed in Table 4. It is noteworthy that, although the warehouse stores 1800 items, the variety of items utilized in each experimental run is significantly less, remaining below 400 even in large-scale instances with 300 orders. When considering the overlap of item types across different orders, the number is lower than 400.

5.2. Comparing the heuristics with optimal results in small-scale instances

If an algorithm process does not include the components of the original ALNS, only ISPDM is used to obtain the corresponding order and tote sequence, then GA is applied to obtain the makespan, which we denote as ISPDM/GA. This section evaluates the performance of the ISPDM/GA and I-ALNS algorithms in small-scale scenarios by solving the MOTSRSP using ISPDM/GA, I-ALNS, and the commercial solver Gurobi. We use the instance parameters in Table 4, randomly generate and run instances five times, with the average value of these five runs taken as the result. Moreover, the time for Gurobi is capped at 1800 s. The experiments are conducted under two distinct workload balance ratios, $\tau = 0$ and 0.2.

Table 5 comprehensively summarizes the results for both small and medium-scale instances. The column labeled Obj_G denotes the average minimum value of the objective function as determined by Gurobi. The columns Obj_S and Obj_A display the average minimum values of the objective function achieved using ISPDM/GA and I-ALNS, respectively. The $Time$ column reflects the average duration required to solve each scenario group with the respective method. The Gap_S column illustrates the discrepancy between the values obtained by ISPDM/GA and those determined by Gurobi, while the Gap_A column highlights the difference between the values obtained by I-ALNS and Gurobi. The specific formula used for these calculations is detailed in the notes section of Table 5. The 'N/A' in Table 5 indicates that it is impossible to identify a feasible solution within 1800 s.

Numerical experiments demonstrate that the I-ALNS exhibits consistent and superior performance across various scenarios. In small-scale instances, specifically SI1, SI2, SI3, and SI4, I-ALNS achieves optimal or near-optimal solutions more swiftly than Gurobi.

Table 6
Evaluating different methods for large-scale instances.

Instance	τ	FCFS/UA		ISPDM/GA			I-ALNS		
		Obj_G	$Time$ (s)	Obj_S	$Time$ (s)	Gap_S (%)	Obj_A	$Time$ (s)	Gap_A (%)
LI1	0	1185.46	0.02	854.80	0.09	−27.88	595.76	54.79	−49.70
	0.2	1036.03	0.02	822.93	0.11	−20.56	573.48	53.21	−44.65
LI2	0	1100.04	0.02	757.48	0.11	−31.14	586.71	71.68	−46.66
	0.2	1231.16	0.02	917.37	0.11	−25.49	613.10	74.93	−50.20
LI3	0	2667.54	0.02	1968.29	0.12	−26.21	1441.41	114.69	−45.69
	0.2	2640.56	0.02	1910.78	0.13	−27.64	1355.94	118.32	−48.64
LI4	0	1798.36	0.02	1333.16	0.14	−25.86	907.89	145.72	−49.51
	0.2	1797.75	0.02	1439.25	0.14	−19.94	958.60	143.11	−46.67

Note. $Gap_S^F = (Obj_S - Obj_F)/Obj_F * 100\%$, $Gap_A^F = (Obj_A - Obj_F)/Obj_F * 100\%$.

Notably, as instance sizes escalate, Gurobi struggles to find optimal solutions within the limited time frame of 1800 s, whereas I-ALNS secures superior solutions in a shorter duration. In medium-scale instances, such as MI1 and MI2, I-ALNS remarkably identifies solutions at least 10.76% more efficient than those discovered by Gurobi, achieving this in just 20 s. In scenarios MI3 and MI4, where Gurobi fails to find feasible solutions, I-ALNS adeptly navigates to high-quality solutions. For the MI4 instance group, the makespan achieved with I-ALNS is reduced by 124.9 s compared to that achieved with ISPDM/GA.

Furthermore, ISPDM/GA consistently arrives at feasible solutions within just 0.1 s. Despite these solutions generally being worse than those produced by Gurobi, the order sequences generated by ISPDM serve effectively as initial solutions for the I-ALNS algorithm. The significant reduction in makespan achieved by I-ALNS, as opposed to ISPDM/GA, underscores the efficacy and superiority of the I-ALNS framework developed in our study.

5.3. Comparing real warehouse operation rules in large-scale instances

In this section, we focus on large-scale scenarios LI1, LI2, LI3, and LI4, comparing our developed heuristic algorithm with a rule-based scheduling method used in an actual enterprise. This method, known as the first-come, first-served (FCFS) rule, sorts orders at the workstation based on their arrival sequence at the warehouse, distributing orders evenly across workstations. Enterprises implement this FCFS sequencing rule to reduce the dwell time of orders in the pool and to streamline the scheduling decision-making process. The tote sequencing approach prioritizes the transportation of totes containing items needed for orders that enter the workstation first. The scheduling of robots follows a uniform assignment rule, meaning that tote sets handling tasks of all workstations are arranged in an integrated sequence, and robots complete the handling of all tote sets according to this sequence.

The integrated sequence is generated by adding the first tote set task of all workstation tote handling sequences to the integrated sequence, traversing all workstations again, and adding the second tote set task of each workstation tote handling sequence to the sequence in turn until all tote handling tasks have been added to this integrated sequence. We refer to this rule-based scheduling method in real warehouses as the FCFS uniform assignment (FCFS/UA) method. We set workstation balance ratios as 0 and 0.2. Different balance ratios have no impact on the FCFS/UA method; under this scheduling rule, the workstation balance ratio is always close to 0.

The numerical experiment results are displayed in Table 6, where the column Gap_A^F shows the gap between the makespan of the I-ALNS and FCFS/UA method, Gap_S^F is the gap between the makespan of ISPDM/GA and FCFS/UA method, and the Obj_F column is the makespan obtained by the FCFS/UA. The note in Table 6 outlines the equations used for these calculations. The numerical experiments indicate that in large-scale scenarios, ISPDM/GA consistently achieves solutions with a smaller makespan compared to FCFS/UA. Specifically, in the LI2 instance group, the reduction in makespan reaches 31.14%. In the LI4 group, with a balance ratio of 0.2, ISPDM/GA reduces the makespan by 19.94% compared to FCFS/UA. I-ALNS maintains its optimal performance across all eight large-scale instance experiments. Compared to FCFS/UA, I-ALNS reduces the makespan by up to 50.2% in the LI2 instance group. The comparison of makespans between I-ALNS and ISPDM/GA reveals that our designed I-ALNS algorithm can significantly enhance the quality of solutions in large-scale numerical experiments.

5.4. Sensitivity analysis

The section further explores sensitivity analysis experiments to determine the impact of specific parameters related to robots, workstations, order characteristics, and warehouse layout on the makespan. Parameters for robots include their number and carrying capacity, while for workstations, parameters include the number of workstations, the capacity of the put wall at the workstation, and the workstation workload balance ratio. This section employs the LI4 instance group and its variants, addressing all scenarios through our proposed I-ALNS algorithm. The findings derive from averaging the solutions across five randomly generated instances, utilizing the parameters of LI4 and its variations.

5.4.1. Impact of the robot settings

The study examines the effects of altering both the number of robots and their buffer capacities on the makespan. The number of robots is gradually increased from a minimum of two to a maximum of ten, with increments of one, alongside adjustments

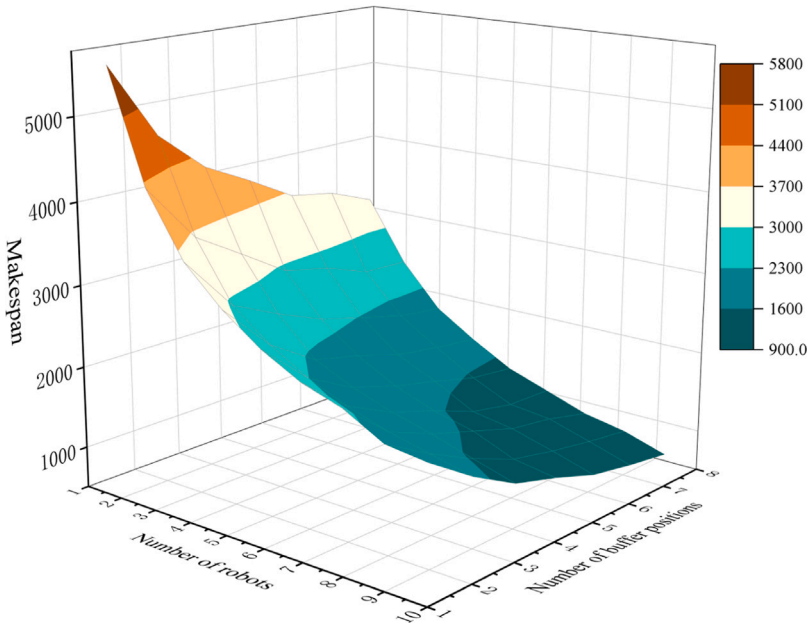


Fig. 5. Effect of the robot number and buffer positions on makespan.

to the buffer capacities on each robot from two to eight, also increasing by one at each stage. The workload balance ratio is maintained at 0.2 across 63 different experimental setups. This methodical exploration allows for detailed observation and analysis of efficiency shifts as indicated by changes in makespan. Fig. 5 displays these experiment outcomes. The experimental results indicate that increasing the number of robots significantly reduces the makespan, particularly when the robot count is high. For example, deploying ten robots diminishes the makespan to less than one-third of that achieved with only 2 robots. Notably, using 2 robots, each equipped with 8 buffer positions, results in a makespan of 3334.85 s. This duration dramatically decreases to 906.9 s when the robot count increases to 10, each with 8 buffer positions. This stark contrast underscores the efficiency enhancements attainable by scaling the number of robots in operational settings.

Additionally, our experiments reveal that increasing the number of buffer positions on robots can swiftly and effectively reduce the makespan. Particularly in the scenarios with 10 robots, the makespan for robots equipped with 8 buffer positions is reduced by 55.4%, comparing robots equipped with just two buffer positions. This highlights that augmenting the number of buffer positions on robots markedly boosts order fulfillment efficiency. However, it is noteworthy that as the number of buffers starts to exceed 5, the effect of further increasing the buffer capacity on reducing the makespan begins to diminish.

In the realm of practical operations, enhancing the buffer capacity of a robot presents itself as a more cost-effective strategy compared to increasing the number of robots in the system. Moreover, operating with a smaller fleet of robots has the added advantage of minimizing potential congestion at workstations. This insight further emphasizes the exceptional capabilities of the MTSR AMR system, where leveraging robots with higher buffer capacities can significantly elevate the overall efficiency of the picking process.

5.4.2. Impact of the workstation settings

The investigation examines the impact of various workstation parameters on the makespan. In this analysis, scenario LI4 is modified by varying the number of workstations, incrementally increasing from 2 up to 10, with each increment adding one more workstation. The capacity of the put wall could be 2, 3, 4, 5, 6, 10, 14, and 18, resulting in 72 unique scenario configurations. Throughout these scenarios, the workstation balance ratio is consistently set at 0.2, offering a uniform comparison basis across different put wall capacities and workstation numbers. This comprehensive evaluation sheds light on how workstation parameters influence system efficiency.

Fig. 6 presents the detailed results of this analysis. Fig. 6(a) shows the makespan for scenarios with smaller put wall capacities (2, 3, 4, 5) with increments of one, while Fig. 6(b) displays the makespan for scenarios with larger put wall capacities (6, 10, 14, 18) and increments of four. Analysis of Fig. 6(a) reveals a consistent pattern: an increase in each put wall capacity by one unit significantly lowers the makespan, particularly at lower capacities (2, 3, 4). Moreover, increasing the number of workstations decreases the makespan in scenarios with put wall capacities of 2, 3, and 4. However, at a capacity of 5, further increases in workstations do not markedly improve picking efficiency. This trend is also reflected in Fig. 6(b), where adding more workstations only marginally reduces the makespan when the put wall capacity reaches 6 or higher. Additionally, it is observed that once the capacity of the put wall hits 10, further enlarging the capacity does not significantly boost order fulfillment efficiency.

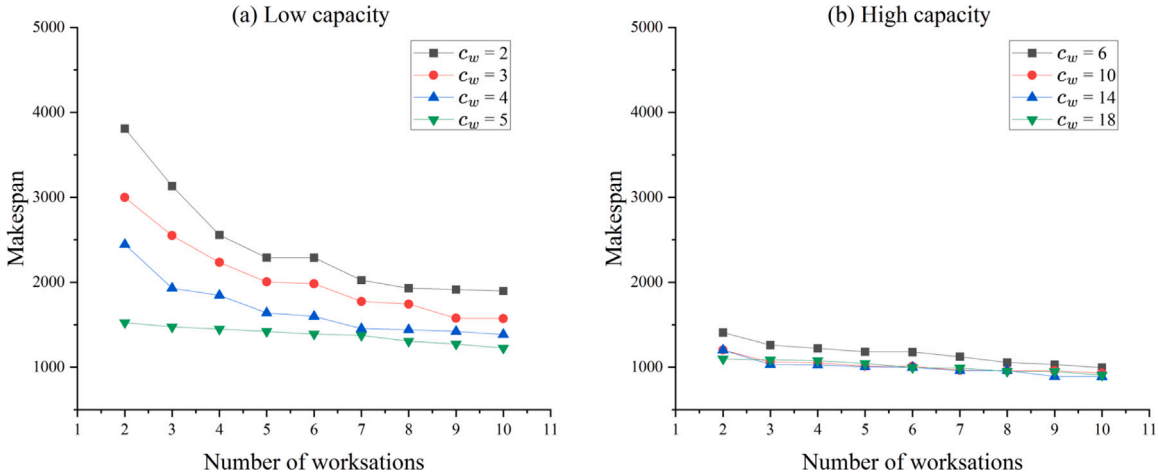


Fig. 6. Effect of workstation number and capacity on makespan.

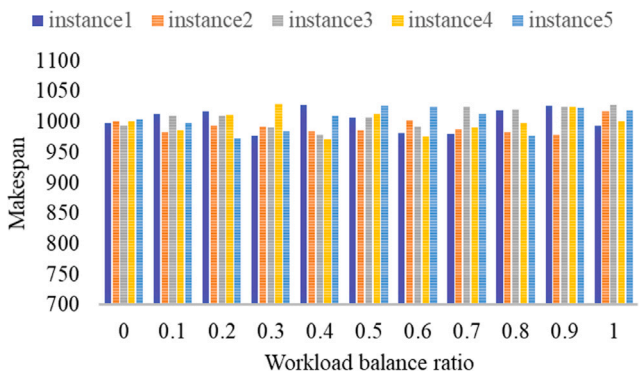


Fig. 7. Effect of workload balance ratio on makespan.

The experiments suggest that although augmenting both the number and capacity of put walls reduces the makespan, the effectiveness of this strategy diminishes after reaching a certain threshold of put wall capacity. Beyond this point, additional increases in the number of workstations or capacity of put walls fail to decrease the makespan substantially.

Furthermore, the impact of the workload balance ratio on the makespan is analyzed by varying the workload balance ratio from 0 to 1 in increments of 0.1, resulting in 11 distinct scenarios. Each scenario contains 5 different randomly generated instances, so there are a total of 55 results in Fig. 7. If the workload balance ratio is equal to 0, orders are evenly assigned to each workstation. When the workload balance is equal to 1, it indicates that some workstations can be assigned orders of up to $2N/K$, as shown in Eq. (8). As depicted in Fig. 7, alterations in the workload balance ratio have a negligible effect on the makespan, which oscillates around 1000 s without displaying a significant or discernible trend across all experimental instances.

This phenomenon is attributed to the intrinsic characteristics of the problem, which focuses on minimizing the makespan. High-quality solutions inevitably involve a more even assignment of orders across the various workstations in large-scale scenarios. As a result, even though a higher workload balance ratio permits some workstations to handle a greater number of orders compared to others, this relaxation of constraints does not necessarily translate into better solutions.

This observation indicates that the optimal strategy is not necessarily about allowing for significant variability in workload across workstations. Instead, ensuring a balanced and equitable order assignment can achieve the optimal or near-optimal scheme. This approach not only preserves overall efficiency but also addresses the uniform management of fatigue levels among human pickers at workstations, contributing to enhanced order fulfillment efficiency and worker satisfaction in the long run.

5.4.3. Impact of the order characteristics

We investigate the impact of order characteristics variations on makespan performance. Based on the LI4, we vary both the total number of orders in a wave and the proportion of single-line, double-line, and triple-line orders. We consider different order quantities (50, 80, 100, 120, 150, 170, and 200), with the proportions of single-line, double-line, and triple-line orders set at five different levels, respectively. The makespan performance is calculated using 10 and 16 robots, totaling 70 different scenarios.

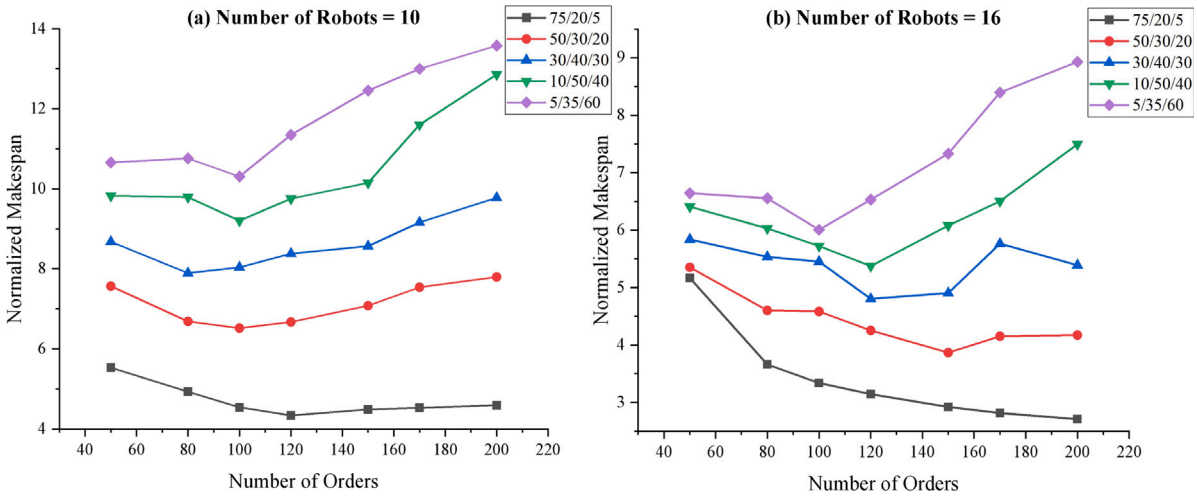


Fig. 8. Effect of the order characteristics on makespan.

To ensure a fair comparison of the makespan under different scenarios with varying total order numbers, we normalized the obtained makespan values by dividing them by the number of orders. A lower normalized makespan indicates a more efficient order fulfillment in that scenario. Fig. 8 presents the experimental results. We derive the following management insights: an increase in the proportion of multi-line orders significantly increases the makespan. When single-line orders dominate the order composition, a larger number of orders in a wave results in a smaller normalized makespan, demonstrating the advantage of unified scheduling. However, increasing the number of orders in a wave does not continuously reduce the normalized makespan in scenarios with a higher proportion of double-line and triple-line orders. In the scenario with 10 robots, better order fulfillment performance is observed with order numbers ranging from 80 to 100, while in the scenario with 16 robots, the highest order fulfillment efficiency is achieved with order numbers between 100 and 120. Regardless of whether 10 or 16 robots are deployed, there is a clear inflection point in the normalized makespan of the wave orders. This phenomenon is because the processing complexity of a large number of multi-line orders is relatively high, making it difficult to find a potential high-quality scheduling scheme.

5.4.4. Impact of the warehouse layout

This section investigates the impact of different warehouse layouts on makespan performance. We ensure that each block contains 600 storage locations, with the number of aisles and storage locations per aisle specified as follows: 6 × 100, 10 × 60, 12 × 50, 15 × 40, and 20 × 30. According to the warehouse dimension data presented in Section 5.1, the width-to-length ratios for each block (considering the horizontal direction as width and the vertical direction as length) are 0.2, 0.6, 0.9, 1.3, and 2.4, respectively. Fig. 9 illustrates the experimental results for all configurations. The experimental analysis reveals that wider warehouse layouts are more efficient than narrow, elongated ones. Specifically, a configuration with 20 aisles per block can reduce the makespan by 26.3% compared to a layout with 6 aisles when processing 200 orders.

6. Conclusion

This paper is devoted to addressing a novel joint optimization problem for the MTSR AMR systems featuring multiple workstations. We develop the MIP model to formulate the MOTSRSP, optimizing key order fulfillment aspects: order processing sequence, tote sequencing based on order diversity at workstations, and efficient robot scheduling for tote transportation to meet order demands. The proposed I-ALNS encompasses a two-stage heuristic approach, which uses the ISPDM to determine the order and tote sequence in the first stage, and then applies GA to obtain the robot scheduling scheme in the second stage. The implementation of the customized I-ALNS algorithm incorporates a destroy-and-repair process to generate a new order sequence, coupled with iterative refinement to enhance solution quality. This pioneering approach offers a comprehensive and optimized solution to the MOTSRSP, advancing the field of MTSR AMR systems.

The numerical experiments cover 12 unique instance groups, representing a spectrum from small to large-scale scenarios. In small-scale instances, a comparison of the I-ALNS algorithm with the Gurobi optimizer reveals that I-ALNS consistently delivers optimal or near-optimal results more efficiently. For large-scale instances, I-ALNS can decrease the makespan by approximately 50.2% relative to a rule-based scheduling approach utilized in an actual warehouse setting, highlighting the efficiency of the developed I-ALNS algorithm.

A sensitivity analysis is also performed, considering variations in the parameters of robots and workstations. Crucial managerial insights are obtained by adjusting the number and capacity of the robots, modifying the number of workstations and the put wall capacities at these workstations, and evaluating the impact of different workload balance ratios on makespan. Based on the aforementioned statements, the following observations are drawn:

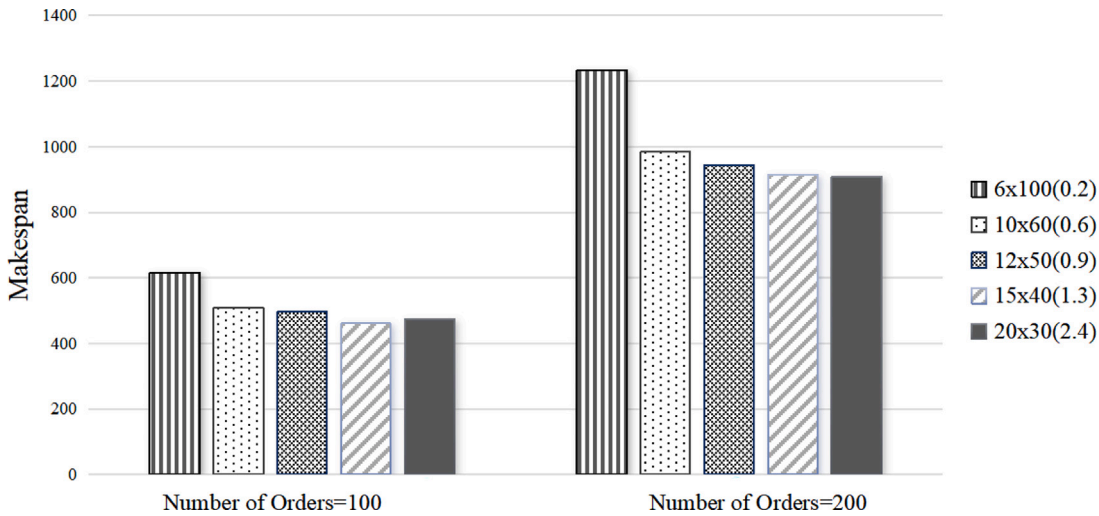


Fig. 9. Effect of the warehouse layout on makespan.

- A marked increase in the number of robots directly correlates with a significant reduction in makespan. Our empirical research reveals that systems equipped with ten robots, each having eight buffer positions, reduce makespan to less than one-third of that in systems with only two robots of the same type. Additionally, enhancing a robot's buffer positions is identified as an effective means to boost efficiency. Configuring systems with ten robots and increasing their buffer positions from two to eight results in a 55.4% reduction in makespan.
- Increasing the number of workstations improves picking efficiency when put wall capacities are limited. However, this benefit plateaus once the capacity threshold reaches five; beyond this point, further expansion in the number of workstations does not effectively reduce the makespan. Furthermore, our research indicates that once the capacity of the put wall at each workstation exceeds ten, any additional increase in both the number of workstations and the put wall capacities at these workstations fails to significantly improve order fulfillment efficiency.
- The workload balance constraint at the workstation has minimal impact on order fulfillment efficiency.
- For varying order characteristics and quantities, setting the number of orders per wave to 100 can improve order fulfillment performance. Furthermore, arranging a wider warehouse layout with a reasonable width-to-length ratio can reduce the makespan by 26.3% compared to a narrow warehouse.

This paper does not specifically consider the volatility of the work efficiency of humans. Therefore, future research can address the challenges posed by human operators at workstations, whose efficiency fluctuates due to fatigue and errors, such as the misplacement of items in order boxes, leading to delays. These challenges are prevalent in real-world applications of MTSR AMR systems. Incorporating the variability of human efficiency into optimization models for joint order fulfillment could significantly enhance the relevance and practical applicability of such studies. Employing a stochastic optimization model to account for uncertainties stemming from fatigue and errors presents a promising direction for improving the expected performance of MTSR AMR systems. Furthermore, in our paper, a tote holds only one type of item. However, MTSR AMR systems can be designed to manage small-volume items, allowing for the storage of diverse categories within a single tote. Strategizing the assortment of items within totes and the order fulfillment process based on this strategy opens new avenues for impactful research. Specifically, researchers can develop a model and adopt a data-driven approach to determine the optimal strategy for item assortment and storage within totes.

CRediT authorship contribution statement

Zhizhen Qin: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Yuexin Kang:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Peng Yang:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 72372088) and Shenzhen Science and Technology Program (Grant No. GJHZ20220913143003006).

Appendix A. Details of genetic algorithm

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.tre.2024.103752.

References

Amazon, 2022. 10 Years of amazon robotics: how robots help sort packages, move product, and improve safety. https://www.aboutamazon.com/news/operations/10-years-of-amazon-robotics-how-robots-help-sort-packages-move-product-and-improve-safety.
Ardjmand, E., Shakeri, H., Singh, M., Sanei Bajgiran, O., 2018. Minimizing order picking makespan with multiple pickers in a wave picking warehouse. Int. J. Prod. Econ. 206, 169–183.
Arslan, B., Ekren, B.Y., 2023. Transaction selection policy in tier-to-tier SBSRS by using deep Q-learning. Int. J. Prod. Res. 61 (21), 7353–7366.
Azadeh, K., De Koster, R., Roy, D., 2019. Robotized and automated warehouse systems: Review and recent developments. Transp. Sci. 53 (4), 917–945.
Boysen, N., Briskorn, D., Emde, S., 2017. Parts-to-picker based order processing in a rack-moving mobile robots environment. European J. Oper. Res. 262 (2), 550–562.
Boysen, N., Schwerdfeger, S., Stephan, K., 2023. A review of synchronization problems in parts-to-picker warehouses. European J. Oper. Res. 307 (3), 1374–1390.
Chen, R., Yang, J., Yu, Y., Guo, X., 2023. Retrieval request scheduling in a shuttle-based storage and retrieval system with two lifts. Transp. Res. E Logist. Transp. Rev. 174, 103117.
De Koster, R., Le-Duc, T., Roodbergen, K.J., 2007. Design and control of warehouse order picking: A literature review. European J. Oper. Res. 182 (2), 481–501.
Ekren, B.Y., Arslan, B., 2022. A reinforcement learning approach for transaction scheduling in a shuttle-based storage and retrieval system. Int. Trans. Oper. Res. 31 (1), 274–295.
Hairobotics, 2023. Cases. https://www.hairobotics.com/cases.
Jiang, M., Leung, K., Lyu, Z., Huang, G.Q., 2020. Picking-replenishment synchronization for robotic forward-reserve warehouses. Transp. Res. E Logist. Transp. Rev. 144, 102138.
Li, Z.P., Zhang, J.L., Zhang, H.J., 2017. Optimal selection of movable shelves under cargo-to-person picking mode. Int. J. Simul. Model. 16 (1), 145–156.
Löffler, M., Boysen, N., Schneider, M., 2022. Picker routing in AGV-assisted order picking systems. INFORMS J. Comput. 34 (1), 440–462.
Matusiak, M., De Koster, R., Saarinen, J., 2017. Utilizing individual picker skills to improve order batching in a warehouse. European J. Oper. Res. 263 (3), 888–899.
Mckinsey, 2021. Insights. https://www.mckinsey.com/industries/retail/our-insights/automation-has-reached-its-tipping-point-for-omnichannel-warehouses.
Menéndez, B., Bustillo, M., Pardo, E.G., Duarte, A., 2017. General variable neighborhood search for the order batching and sequencing problem. European J. Oper. Res. 263 (1), 82–93.
Muter, I., Öncan, T., 2015. An exact solution approach for the order batching problem. IIE Trans. 47 (7), 728–738.
Muter, I., Öncan, T., 2022. Order batching and picker scheduling in warehouse order picking. IISE Trans. 54 (5), 435–447.
Petersen, C.G., Schmenner, R.W., 1999. An evaluation of routing and volume-based storage policies in an order picking operation. Decis. Sci. 30, 481–501.
Protalinski, E., 2019. Shopify acquires 6 river systems for $450 million to expand its AI-powered fulfillment network. https://venturebeat.com/commerce/shopify-acquires-6-river-systems-for-450-million-to-expand-its-ai-powered-fulfillment-network.
Qin, Z., Yang, P., Gong, Y., De Koster, R., 2024. Performance analysis of multi-tote storage and retrieval autonomous mobile robot systems. Transp. Sci. online.
Ratliff, H.D., Rosenthal, A.S., 1983. Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem. Oper. Res. 31 (3), 507–521.
Røpke, S., Pisinger, D., 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp. Sci. 40, 455–472.
Roy, D., Krishnamurthy, A., Heragu, S.S., Malmborg, C.J., 2012. Performance analysis and design trade-offs in warehouses with autonomous vehicle technology. IIE Trans. 44 (12), 1045–1060.
Tappia, E., Roy, D., De Koster, R., Melacini, M., 2017. Modeling, analysis, and design insights for shuttle-based compact storage systems. Transp. Sci. 51 (1), 269–295.
Valle, C.A., Beasley, J.E., 2021. Order allocation, rack allocation and rack sequencing for pickers in a mobile rack environment. Comput. Oper. Res. 125, 105090.
Wang, Z., Sheu, J.-B., Teo, C.-P., Xue, G., 2022. Robot scheduling for mobile-rack warehouses: Human–robot coordinated order picking systems. Prod. Oper. Manage. 31 (1), 98–116.
Wang, B., Yang, X., Qi, M., 2023. Order and rack sequencing in a robotic mobile fulfillment system with multiple picking stations. Flex. Serv. Manuf. J. 39 (2), 509–547.
Xiang, X., Liu, C., Miao, L., 2018. Storage assignment and order batching problem in Kiva mobile fulfilment system. Eng. Optim. 50 (11), 1941–1962.
Xie, L., Thieme, N., Krenzler, R., Li, H., 2021. Introducing split orders and optimizing operational policies in robotic mobile fulfillment systems. European J. Oper. Res. 288 (1), 80–97.
Yang, J., De Koster, R., Guo, X., Yu, Y., 2023. Scheduling shuttles in deep-lane shuttle-based storage systems. European J. Oper. Res. 308 (2), 696–708.
Yang, X., Hua, G., Hu, L., Cheng, T., Huang, A., 2021. Joint optimization of order sequencing and rack scheduling in the robotic mobile fulfillment system. Comput. Oper. Res. 135, 105467.
Zhang, G., Gao, L., Shi, Y., 2011. An effective genetic algorithm for the flexible job-shop scheduling problem. Expert Syst. Appl. 38 (4), 3563–3573.
Zhen, L., Tan, Z., De Koster, R., Wang, S., 2023. How to deploy robotic mobile fulfillment systems. Transp. Sci. 57 (6), 1671–1695.
Zhuang, Y., Zhou, Y., Yuan, Y., Hu, X., Hassini, E., 2022. Order picking optimization with rack-moving mobile robots and multiple workstations. European J. Oper. Res. 300 (2), 527–544.
Zou, B., De Koster, R., Xu, X., 2018a. Operating policies in robotic compact storage and retrieval systems. Transp. Sci. 52 (4), 788–811.
Zou, B., Gong, Y.Y., Xu, X., Yuan, Z., 2017. Assignment rules in robotic mobile fulfilment systems for online retailers. Int. J. Prod. Res. 55 (20), 6175–6192.
Zou, B., Xu, X., Gong, Y.Y., De Koster, R., 2016. Modeling parallel movement of lifts and vehicles in tier-captive vehicle-based warehousing systems. European J. Oper. Res. 254 (1), 51–67.
Zou, B., Xu, X., Gong, Y.Y., De Koster, R., 2018b. Evaluating battery charging and swapping strategies in a robotic mobile fulfillment system. European J. Oper. Res. 267 (2), 733–753.
Žulj, I., Kramer, S., Schneider, M., 2018. A hybrid of adaptive large neighborhood search and tabu search for the order-batching problem. European J. Oper. Res. 264 (2), 653–664.
Žulj, I., Salewski, H., Goeke, D., Schneider, M., 2022. Order batching and batch sequencing in an AMR-assisted picker-to-parts system. European J. Oper. Res. 298 (1), 182–201.